A Survey on Deep Neural Network Pruning: Taxonomy, Comparison, Analysis, and Recommendations

Hongrong Cheng, Miao Zhang, *Member, IEEE*, and Javen Qinfeng Shi, *Member, IEEE*

(Survey Paper)

***Abstract*—Modern deep neural networks, particularly recent large language models, come with massive model sizes that require significant computational and storage resources. To enable the deployment of modern models on resource-constrained environments and to accelerate inference time, researchers have increasingly explored pruning techniques as a popular research direction in neural network compression. More than three thousand pruning papers have been published from 2020 to 2024. However, there is a dearth of up-to-date comprehensive review papers on pruning. To address this issue, in this survey, we provide a comprehensive review of existing research works on deep neural network pruning in a taxonomy of 1) universal/specific speedup, 2) when to prune, 3) how to prune, and 4) fusion of pruning and other compression techniques. We then provide a thorough comparative analysis of eight pairs of contrast settings for pruning (e.g., unstructured/structured, one-shot/iterative, data-free/data-driven, initialized/pre-trained weights, etc.) and explore several emerging topics, including pruning for large language models, vision transformers, diffusion models, and large multimodal models, post-training pruning, and different levels of supervision for pruning to shed light on the commonalities and differences of existing methods and lay the foundation for further method development. Finally, we provide some valuable recommendations on selecting pruning methods and prospect several promising research directions for neural network pruning. To facilitate future research on deep neural network pruning, we summarize broad pruning applications (e.g., adversarial robustness, natural language understanding, etc.) and build a curated collection of datasets, networks, and evaluations on different applications. We maintain a repository on https://github.com/hrcheng1066/awesome-pruning that serves as a comprehensive resource for neural network pruning papers and corresponding open-source codes. We will keep updating this repository to include the latest advancements in the field.**

Received 1 May 2023; revised 11 June 2024; accepted 10 August 2024. Date of publication 21 August 2024; date of current version 5 November 2024. The work of Miao Zhang was partially sponsored in part by the National Natural Science Foundation of China under Grant 62306084 and under Grant U23B2051, and in part by Shenzhen College Stability Support Plan under Grant GXWD20231128102243003. Recommended for acceptance by W. Hu. *(Hongrong Cheng and Miao Zhang contributed equally to this work.) (Corresponding author: Miao Zhang.)*

Hongrong Cheng and Javen Qinfeng Shi are with the University of Adelaide, Adelaide, SA 5005, Australia (e-mail: hongrong.cheng@adelaide.edu.au; javen.shi@adelaide.edu.au).

Miao Zhang is with the Harbin Institute of Technology, Shenzhen 518055, China (e-mail: zhangmiao@hit.edu.cn).

This article has supplementary downloadable material available at https://doi.org/10.1109/TPAMI.2024.3447085, provided by the authors.

Digital Object Identifier 10.1109/TPAMI.2024.3447085

***Index Terms*—Deep neural network pruning, model compression, model acceleration, large language models, vision transformers, large multimodal models, diffusion models, edge devices.**

I. Introduction

OVER the past several years, Deep Neural Networks (**DNNs**) have achieved conspicuous progress in various domains and applications, such as Computer Vision (**CV**) [1], [2], Natural Language Processing (**NLP**) [3], [4], Audio Signal Processing (**ASP**) [5], [6], and cross-modal applications [7], [8]. Although DNNs achieve remarkable success in various areas, their performance relies heavily on model parameters and computational cost. For example, the widely used ResNet-50 [9] takes over 95MB of memory, containing over 23 million parameters [10]. $\text{BERT}_{\text{BASE}}$ [3] is around 440MB with 110 million parameters, GPT-3 includes up to 175 billion parameters [11], and GPT-4 has even more. The trend of enlarging neural network size is anticipated to persist.

However, the more parameters of DNNs, the more time and memory space they typically require for processing the inputs [12]. The high training and inference costs associated with these models present a significant challenge to their deployment on devices constrained by limited computational resources (such as CPU, GPU, and memory), energy, and bandwidth [13], [14], [15]. For example, real-life applications such as autonomous driving, field rescue, and bushfire prevention necessitate high accuracy and efficient resource usage, including fast real-time response and compact memory footprint. Deep neural networks' computational complexity and memory footprint can make them impractical for deployment on edge devices [16]. With the popularity of Large Language Models (**LLMs**) in recent years, there is growing interest in compressing neural networks for computers with flexible hardware requirements [17]. In addition, deep neural networks that contain redundant features can undermine their robustness, elevating the risk of adversarial attacks [18]. For instance, high-dimensional feature spaces created by these networks can provide more entry points for adversarial attacks, undermining the network's ability to generalize beyond its original training data.

To relieve this issue, researchers have proposed various neural network compression techniques to design lightweight models,

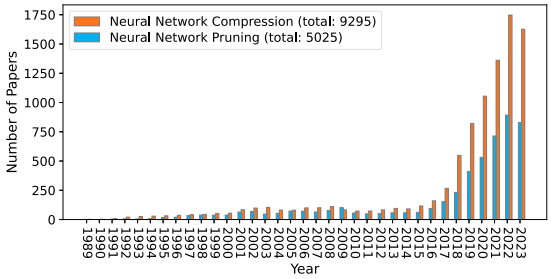


Fig. 1. The number of papers on neural network pruning and compression from 1989-2023.[1]

TABLE I
REPRESENTATIVE SURVEYS AND DESCRIPTIONS

Survey	Main work description	Year
[41]	survey of pruning methods	1993
[42]	overview 81 papers and propose ShrinkBench, an open-source framework for evaluation	2020
[43]	pruning+weight sharing+low-rank matrix+ KD+quantization	2020
[44]	pruning criteria+procedure	2020
[45]	pruning+quantization+KD+low-rank factor.	2020
[32]	pruning+KD+NAS+Tensor-Decompose +quantization+hardware acceleration	2020
[46]	work on sparsity in deep learning up to 2020	2021
[36]	overview pruning at initialization	2022
[33]	pruning+KD+NAS+quantization +Tensor-Decompose+hardware accel.	2022
[37]	structured pruning	2023
[40]	pruning+KD+quantization+others for LLM	2023
[47]	pruning+KD+quantization+others for LLM	2024
[34]	pruning+KD+quantization+others for LLM	2024
[35]	model compression+others for LLMs	2024

including neural network pruning ([19], [20]), low-rank factorizations of the weight matrices ([21], [22]), quantization ([23], [24]), knowledge distillation ([25], [26]), neural architecture search ([27], [28]), and other techniques ([29], [30]). Among them, there is continuing interest in neural network pruning, which has been proven as a desirable and effective way to save memory space and computation time at inference while maintaining a comparable or even better performance compared to the original DNNs. As shown in Fig. 1, the number of papers on pruning has been markedly increasing since 2015. It presents more than half of the papers on neural network compression.

Research on pruning can be traced back to literature as early as 1988 [31]. However, it was only until the emergence of [13] that the research community realized the potential of pruning in removing significant redundancy in deep neural networks, and pruning began to gain widespread attention. Several pieces of literature review prior work on deep neural network pruning, as shown in Table I. Although these works overview several aspects of pruning and provide helpful guidance for researchers, many of them (e.g., [32], [33], [34], [35]) focus on multiple compression techniques, such as pruning, quantization, and knowledge distillation, with only brief examination of each technique. For example, Mishra et al. [32] summarize compression techniques, including pruning, quantization, low-rank factorization, and knowledge distillation, where pruning is primarily introduced from channel/filter pruning, and many essential pruning techniques (such as lottery ticket hypothesis) are not included. Some reviews concentrate on one specific aspect. For example, Wang et al. [36] provide only an overview of pruning at initialization and do not include studies on pruning during training, pruning after training, etc. He and Xiao [37] focuses only on structured pruning and does not discuss the other two common types of pruning: unstructured and semi-structured pruning. Ghimire et al. [33]) focus on reviewing Convolutional Neural Network (**CNN**) pruning and lack descriptions of pruning for other deep neural networks, such as Recurrent Neural Networks (**RNNs**) [38], Transformer based models [17], and diffusion models [39]. Some recent review works (such as [35], [40]) survey compression techniques for large models, encompassing a broad array of topics from pruning, quantization, knowledge distillation, etc. Among these, pruning is discussed concisely.

This survey aims to provide a comprehensive overview of deep neural network pruning for diverse readers. We review representative pruning methods, propose a new taxonomy, conduct a comprehensive analysis of how different pruning techniques perform in practice, and give practitioners who wish to utilize pruning recommendations on choosing a suitable pruning method for various requirements. Our contributions are as follows:

1) Comprehensive review. To our best knowledge, this survey is the most comprehensive overview of modern deep neural network pruning techniques. It distills ideas from over 300 related academic papers, covering pruning methods for small to medium models to cutting-edge large models. In addition, we establish a new taxonomy, as shown in Fig. 2, and provide detailed descriptions of the representative methods for each class of pruning methods.

2) Comparative experiments and analysis. We conduct a comparative analysis of eight pairs of contrast settings for pruning and emerging advances, such as pruning for LLMs and different supervision levels for pruning. Unlike existing surveys, we conduct experiments and provide discussions.

3) Collection of abundant resources. We summarize miscellaneous pruning applications and provide benchmark datasets, networks, and evaluations for different applications. Our collected resources in Appendix D, available online, can guide researchers and practitioners in understanding, utilizing, and developing different network pruning methods for various requirements. Ongoing updates of representative pruning efforts are available at https://github.com/hrcheng1066/awesome-pruning.

4) Recommendations and future directions. This survey provides valuable recommendations for choosing appropriate pruning methods for different application requirements and highlights promising future research directions.

The remainder of this survey is organized as follows. In Section II, we establish a clear taxonomy of pruning. Sections III–VI offer an overview of speedup, when to prune, and how to prune, followed by a comprehensive comparative analysis of different kinds of pruning methods in Section VII. Section VIII discusses integrating pruning with other compression methods. Practical recommendations for choosing pruning methods and future directions are provided in Section IX. We conclude this paper in Section X.

[1]The data is from https://www.webofknowledge.com.

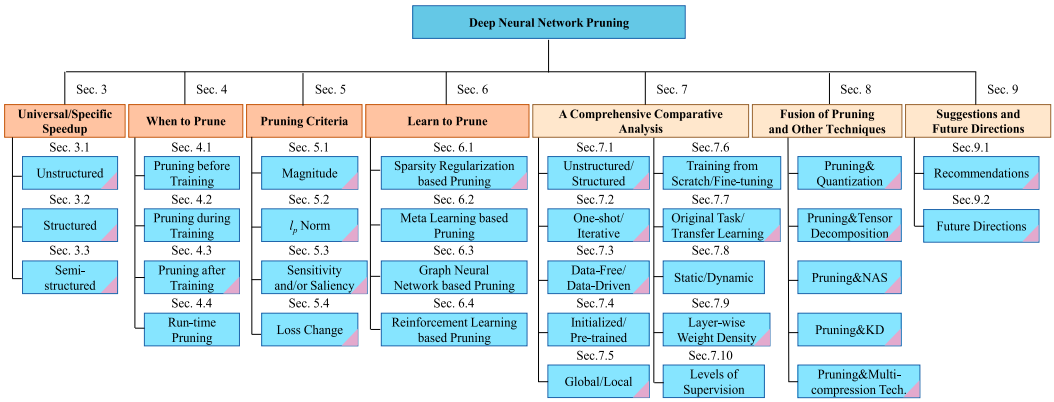


Fig. 2. An overview of the hierarchical structure of the survey, with sections involving large models highlighted by a pink triangle.

II. Taxonomy

There are three critical questions when pruning a deep neural network. *(1) Whether to achieve universal or specific acceleration through neural network pruning? Universal acceleration operates independently of specialized hardware/software, while specific acceleration relies on it. (2) When to prune the neural network? Specifically, is the neural network pruned before, during, or after training the network for static pruning or dynamic (i.e., run-time) pruning? (3) Whether to prune based on specific criteria or learn to prune?* The answers to the three questions correspond to the three primary aspects of deep neural network pruning, as depicted in the orange sections of Fig. 2.

The first question is whether speedup depends on specific hardware/software. It is usually divided into three types: unstructured ([48], [49], [50]), semi-structured (also called pattern-based) ([17], [51], [52]) and structured ([20], [39], [53]). Only structured pruning can achieve universal neural network acceleration without requiring special hardware or software. Conversely, both unstructured and semi-structured pruning need the support of special hardware or software. Given that the primary objective of pruning is acceleration, the first question addresses the most fundamental and user-concerned aspect of this process.

The second question particularly concerns the arrangement between pruning weights and training weights of the neural network for static pruning. Based on whether pruning is performed before, during, or after training the network, static pruning can be divided into three categories: pruning before training (**PBT**) ([50], [54], [55], [56]), pruning during training (**PDT**) ([57], [58], [59]), and pruning after training (**PAT**) ([20], [48], [60]). In dynamic pruning, subnetworks are generated at run-time for each input data.

The third question considers whether to prune neural networks using specific criteria or by learning. Criteria rely on a scoring formula to measure the importance of each weight (or filter, neuron, and so on). Commonly used pruning criteria include magnitude, norm, loss change, etc. In addition, it is possible to prune neural networks by learning, such as through sparsity regularization training [59] or dynamic sparse training [61]. Whether through criteria or learning, pruning aims to determine the weights of a network that should be pruned.

The above three aspects determine the main characteristics of a pruning algorithm. Different combinations of these three aspects form various pruning methods. We provide a new taxonomy of deep neural network pruning in Sections III–VI, and Section VIII, as shown in Fig. 2.

III. Specific or Universal Speedup

This section categorizes pruning into unstructured, semi-structured, and structured. The first two types correspond to specific speedup, while the third corresponds to universal speedup.

A. Unstructured Pruning

Unstructured pruning, also called non-structured pruning or weight-wise pruning, is the finest-grained case.

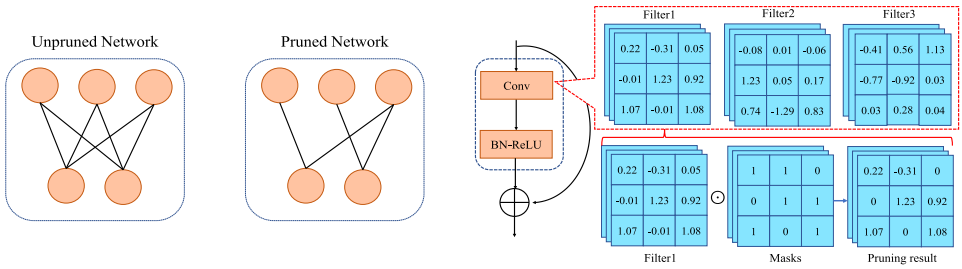
Definition 1 (Unstructured Pruning). Given neural network weights $\mathbf{w} = \{w_0, w_1, \ldots, w_K\}$, a dataset $\mathcal{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ composed of input $(\mathbf{x}_i)$, output $(\mathbf{y}_i)$ pairs, and a desired total number of non-zero weights k (less than K), unstructured pruning can be written as the following constrained optimization problem [54]:

$$\min_{\mathbf{w}} \ \mathcal{L}(\mathbf{w}; \mathcal{D}) = \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{w}; (\mathbf{x}_i, \mathbf{y}_i)),$$
$$\text{s.t. } \|\mathbf{w}\|_0 \leq k. \tag{1}$$

In practice, for small or medium models, unstructured pruning usually does not directly set the weights to 0 but sets their corresponding masks (or indicators) $\mathbf{m}$ to 0 [56], [62]. In this case, unstructured pruning is regarded as applying a binary mask to each weight. Consequently, (1) is correspondingly changed as

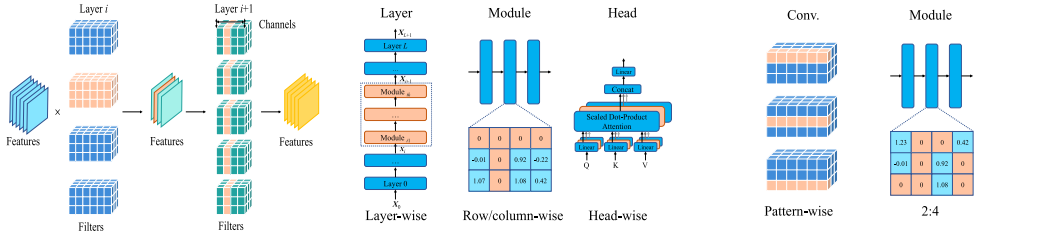
$$\min_{\mathbf{w},\mathbf{m}} \ \mathcal{L}(\mathbf{w} \odot \mathbf{m}; \mathcal{D}) = \min_{\mathbf{w},\mathbf{m}} \frac{1}{N} \sum_{i=1}^{N} \ell(\mathbf{w} \odot \mathbf{m}; (\mathbf{x}_i . \mathbf{y}_i)),$$
$$\text{s.t. } \|\mathbf{m}\|_0 \leq k. \tag{2}$$

Generally, the network is retrained (i.e., fine-tuning or training-from-scratch) with fixed masks $\mathbf{m}$, and the masked-out weights are not involved in retraining. In the case of large models, such as LLMs, assigning a mask to each weight poses a challenge due to



(a) From the view of neurons and connections (b) From the view of weights and masks

Fig. 3. The visualization of unstructured pruning. The light orange circles denote neurons.



(a) Structured pruning for CNNs (b) Structured pruning for Transformers (c) Examples of semi-structured pruning

Fig. 4. The visualization of typical structured and semi-structured pruning: each Conv. consists of cubes representing weights. Orange cube sets/blocks indicate the pruned structures. Best view in color and zoom in.

the immense number of weights. It is common to directly set the weights that need to be pruned to zero, as seen in [17], [49]. Fig. 3 is an example of weight-wise pruning by removing the weights directly (as shown in Fig. 3(a)) or masking the weights with their corresponding masks (as shown in Fig. 3(b)), respectively. Since it can remove weights anywhere, the irregular replacement of non-zero weights leads to actual acceleration requiring the support of special software and/or hardware [13], [58], [63], [64]. Therefore, we classify unstructured pruning as a specific speedup technique.

B. Structured Pruning

Definition 2 (Structured Pruning). Given a specific prune ratio and a neural network with $S = \{s_1, s_2, \ldots, s_L\}$, where s_i can be the set of channels, filters, neurons, or transformer attention heads in layer i. Structured pruning aims to search for $S' = \{s'_1, s'_2, \ldots, s'_L\}$ to minimize performance degeneration and maximize speed improvement under the given prune ratio, where $s'_i \subseteq s_i, i \in \{1, .., L\}$.

Structured pruning removes entire filters ([15]), channels ([19], [62]), transformer attention heads ([20]), or even layers ([65]), as shown in Fig. 4(a) and (b), and can rebuild a narrow model with a regular structure.[2] It does not require the support of special hardware and software (such as sparse convolution libraries) and can directly speed up networks and reduce the size of the neural networks [16], [62], [63], [67].

[2]For Transformers [66], deleting rows (columns) of weight matrices is similar to pruning filters (channels) for CNNs.

C. Semi-Structured Pruning

To improve the flexibility of structured pruning and achieve a lower accuracy drop when the pruning rate is high, some works ([51], [52]) introduce semi-structured pruning, also called pattern-based pruning in [52], to achieve high accuracy and structural regularity simultaneously. Some examples are shown in Fig. 4(c). For example, Meng et al. [51] treat a filter as several stripes and propose to prune stripes in each filter. SparseGPT [17] introduces a 2:4 or 4:8 sparsity pattern to reduce a LLM's parameters by half. In this configuration, at least two zeros are mandatory in every set of four consecutive values. The 2:4 pattern can utilize NVIDIA Ampere GPU's sparse tensor cores to accelerate matrix multiplication [60]. In contrast, structured pruning is classified as coarse-grained structured pruning ([52], [68]), while semi-structured pruning is classified as fine-grained.

IV. When to Prune

This section distinguishes three pipelines for static pruning, as illustrated in Fig. 5, and run-time pruning. The examples of the three pipelines of static pruning are shown in Fig. 6. Some notations and statistics on three static pruning pipeline types are shown in Appendix A and D, respectively, available online.

A. Pruning Before Training

Pruning Before Training (**PBT**) (as shown in Table II), also called foresight pruning [56] or pruning at initialization [50], [54], represents a class of pruning methods that use randomly

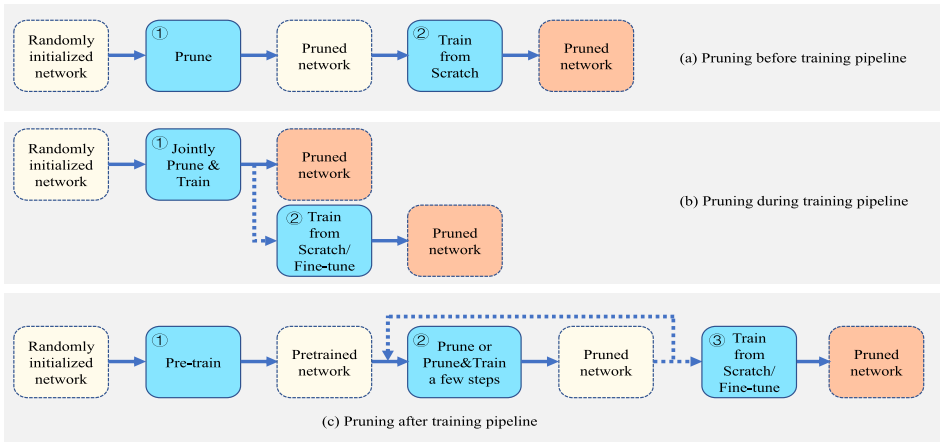
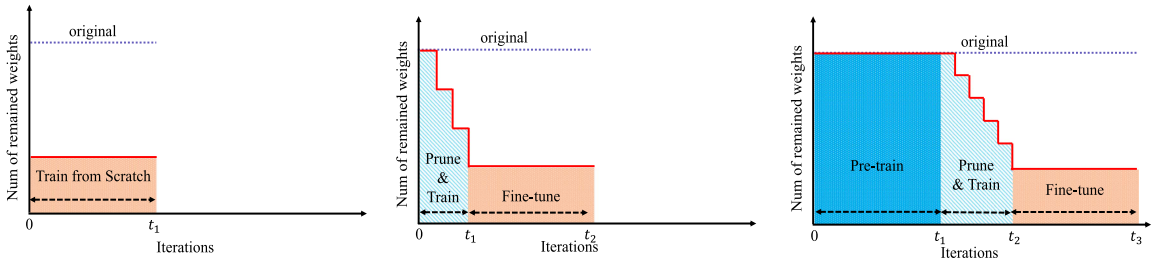


Fig. 5. The typical pipelines of static pruning. Dashed boxes represent models and solid boxes denote actions. Dashed arrows indicate optional steps.



(a) SNIP [54] as an example of PBT (b) FCF [89] as an example of PDT (c) GFP [62] as an example of PAT

Fig. 6. The illustration of typical sparsity changes for PBT, PDT, and PAT (best view in color and zoom in). PBT is usually the cheapest, while PAT is the most expensive.

TABLE II
REPRESENTATIVE METHODS OF PRUNING BEFORE TRAINING

Method	Criteria	U/S	Data-Driven (Y/N)	One-shot (Y/N)
SNIP (2019) [54]	$\lvert\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w}) \odot \mathbf{w}\rvert$	U	Y	Y
GraSP (2020) [56]	$-(H\nabla_{\mathbf{w}}\mathcal{L}(\mathbf{w})) \odot \mathbf{w}$	U	Y	Y
Smart-Ratio (2020) [55]	keep-ratios	U	N	Y
SynFlow (2020) [50]	$\nabla_w \mathcal{R}_{SF}(\mathbf{w}) \odot \mathbf{w}$	U	N	N
PFS (2020) [70]	l_1-norm based sparsity regularization	S	Y	N
RST (2022) [71]	l_2-norm based sparsity regularization	U	Y	N

"U/S" denotes unstructured or structured pruning.[3]

initialized weights for pruning the network. The principal motivation of PBT methods is to eliminate the cost of pre-training. Without loss of generality, we define a neural network as a function $f(\mathbf{x}; \mathbf{w} \odot \mathbf{m})$. The mask $\mathbf{m}$ is used for pruning initialized weights $\mathbf{w}_0$ sampled from a specific initialization distribution. After pruning, the network $f(\mathbf{x}; \mathbf{w}_0 \odot \mathbf{m}')$ is trained to converge to $f(\mathbf{x}; \mathbf{w}_T \odot \mathbf{m}')$ after T epochs, where $\mathbf{m}'$ indicates the sparsity after pruning.

PBT usually follows two steps: directly prunes untrained dense network based on a specific criterion and then trains the sparse network to convergence for high performance, as illustrated in Fig. 5(a). The second step is related to static sparse training [69] that aims to train a sparse network with a fixed sparse pattern. By avoiding the time-consuming pre-training process, PBT methods achieve the gains at training and inference time.

Currently, PBT primarily targets CNNs. Lee et al. [54] pioneer the research of PBT and propose a Single-shot Network Pruning (**SNIP**) method to remove the weights whose absence leads to the slightest loss change. Lee et al. [72] explain the feasibility of SNIP through signal propagation, empirically find pruning damages the dynamical isometry [73] of neural networks, and propose a data-free orthogonal initialization, an approximation of exact isometry, to prune random networks. Different from the signal propagation perspective in [72], which focuses on initialization scheme, Wang et al. [56] propose Gradient Signal Preservation (**GraSP**) to exploit gradient norm after pruning to remove weights that have the least effect on the gradient flow. Tanaka et al. [50] go a step further and point out that an effective

[3]See Appendix A, available online, for the notations in Tables II–IV.

TABLE III
REPRESENTATIVE METHODS OF PRUNING DURING TRAINING

Method	Object Function/Criteria	Retrain (Y/N)	U/S
Network Slimming (2017) [57]	$\min_{\mathbf{w},\boldsymbol{\gamma}} \ell(\mathbf{y}, f(\mathbf{x};\mathbf{w})) + \lambda\|\boldsymbol{\gamma}\|_1$	Y	S
SSS (2018) [59]	$\min_{\mathbf{w},\boldsymbol{\gamma}} \ell(\mathbf{y}, f(\mathbf{x};\mathbf{w},\boldsymbol{\gamma})) + \mathcal{R}(\mathbf{w}) + \lambda\|\boldsymbol{\gamma}\|_1$	N	S
SET (2018) [91]	$\|\mathbf{w}\|$ for drop and random for grow	N	U
DSA (2020) [85]	$\min_{\mathcal{A}} \mathcal{L}_v(\mathbf{w}(\mathcal{A}), \mathcal{A}))$	N	S
GraNet (2021) [92]	$\|\mathbf{w}\|$ for drop and gradient for grow	N	U&S
FreeTickets (2022) [93]	$\|\mathbf{w}\|$ for drop and gradient for grow	N	U

"Retrain" refers to training from scratch or fine-tuning, "U/S" denotes unstructured or structured pruning.

subnetwork can be identified without training and looking at the data. They propose a data-free pruning algorithm called Iterative Synaptic Flow Pruning (**SynFlow**) that uses the concept of synaptic flow to consider the interaction of different layers and avoid layer collapse through gradual pruning. Gebhart et al. [74] exploit the Path Kernel (i.e., the covariance of path values in a network) based on Neural Tangent Kernel [75] to unify several initialization pruning methods, such as SNIP [54], CraSP [56], SynFlow [50], under a single framework. Some works ([55], [76]) suggest that the delicate pruning criteria may not be crucial for performance, but the layerwise sparsity. For example, Su et al. [55] propose randomly pruning each layer by using a series of layerwise keep-ratios (smart-ratios) without using any data.

Inspired by Weight Agnostic Neural Networks (**WANNs**) [77], Ramanujan et al. [78] empirically find that with an untrained network grows wider and deeper, it will contain a subnetwork that performs as well as a trained network with the same number of parameters. Then they propose the edge-popup method to identify such randomly initialized subnetworks. Unlike this method, which maintains randomly weighted values, Bai et al. [71] propose Dual Lottery Ticket Hypothesis (**DLTH**) where both the subnetwork and weights are randomly selected at initialization and propose Random Sparse Network Transformation (**RST**) which fixes the sparse architecture but gradually train the remaining weights.

Some recent works ([69], [70], [79]) explore why an effective subnetwork can be identified at initialization. For example, Liu et al. [69] empirically demonstrate that the network size and appropriate layer-wise prune ratios are two vital factors in training a randomly pruned network at initialization from scratch to match the performance of the dense models. In contrast, Kumar et al. [80] find that at the same prune ratio, subnetworks derived by pruning during or after training exhibit higher effective parameter count and greater expressiveness compared to those pruned at initialization.

B. Pruning During Training

Pruning During Training (**PDT**) (as shown in Table III) generally takes a randomly initialized dense network $f(\mathbf{x};\mathbf{w}_0)$ as the input model and jointly trains and prunes the neural network by updating weights $\mathbf{w}$ and masks of weights (or filters, channels, etc.) $\mathbf{m}$ during training. These dynamic schemes change the masks and produce the subnetwork $f(\mathbf{x};\mathbf{w}_t \odot \mathbf{m}_t)$ after t iterations/epochs. After pruning, many PDT methods ([59], [81], [82], [83], [84], [85], [86]) directly obtain the subnetworks without the need for training-from-scratch or fine-tuning. The typical pipeline of PDT is illustrated in Fig. 5(b). PDT methods have been less explored due to the more complicated dynamic process than PBT and PAT methods. We summarize the main prior solutions into four paradigms: (1) sparsity regularization based, (2) dynamic spare training based, (3) score-based, and (4) differentiable pruning based. Methods related to (2) conduct sparse-to-sparse training, while the other three take dense-to-sparse training.

1) Sparsity Regularization Based Methods: Sparsity regularization technique [58] is commonly used in PDT methods. This category of methods starts with dense networks, imposes sparse constraints on loss functions, and usually zeros out some weights or their masks during training. The main effort is to design the effective target loss function $\mathcal{L}$ with an advanced penalty scheme and efficient optimization algorithms. For example, Wen et al. [58] propose Structured Sparsity Learning (**SSL**) to learn a sparse structure by group LASSO [87] regularization during training. However, SSL requires computing the gradients of the regularization term w.r.t. all the weights, which is non-trivial. Gordon et al. [88] propose **MorphNet** that reuses the parameters of Batch Normalization (**BN**) and conducts sparsity regularization on these parameters. However, some networks (e.g., certain VGGNets [1]) have no BN layers. Instead of reusing BN parameters, some works associate scaling factors with channels, layers, etc. For example, Huang and Wang [59] propose Sparse Structure Selection (**SSS**) that associates scaling factors for CNN micro-structures (e.g., channels, residual blocks) and exploit sparsity regularization to force the output of the micro-structures to zero, rather than pushing the weights in the same group to zero as in [58]. In addition, SSS does not require extra fine-tuning in [58]. Li et al. [89] propose Factorized Convolutional Filter (**FCF**) which introduces a binary scalar to each filter and proposes a back-propagation algorithm with Alternating Direction Method of Multipliers (**ADMM**) [90] to train the weights and the scalars during training jointly.

2) Dynamic Sparse Training Based Methods: A class of the PDT methods ([81], [91], [92], [94], [95], [96], [97]) take randomly initialized sparse network rather than dense network as the input model. Subsequently, a common method involves pruning a fraction of unimportant weights and then regrowing the same number of new weights to adjust the sparse architecture. By repeating the prune-and-grow cycle during training, these method keeps searching for better sparse architecture, a process classified as dynamic sparse training in [81].

For example, Mocanu et al. [91] propose Sparse Evolutionary Training (**SET**) that removes the smallest positive and most negative weights and grows new weights in random locations. Instead of pruning a fixed fraction of weights at each redistribution step, such as in SET [91], Mostafa and Wang [95]

TABLE IV
REPRESENTATIVE METHODS OF PRUNING AFTER TRAINING

Method	Object Function/Criteria	U/S
Auto-Balance (2018) [105]	score($\mathcal{F}_{i,j}$)=$\|\mathcal{F}_{i,j}\|_1$	S
LTH (2019) [48]	score(w_i)=$\|w_i\|_1$	U
Taylor-FO-BN (2019) [106]	score(w_i)=$(\nabla_{w_i}\ell \odot w_i)^2$	S
SSR (2019) [107]	$\min_{\mathbf{w}} \frac{1}{N}\sum_{i=1}^{N}\ell(y_i, f(x_i,\mathbf{w})) + \lambda\mathcal{R}(\mathbf{w})$	S
GReg (2021) [108]	score($\mathcal{F}_{i,j}$)=$\|\mathcal{F}_{i,j}\|_1$	U&S
GFP (2021) [62]	score(c_i)=$\sum_{a=1}^{N}(\nabla_{m_i}\ell_a)^2$ or score(c_i)=$\sum_{a=1}^{N}(\sum_{g\in Group}\nabla_{m_i^g}\ell_a)^2$	S
SparseGPT (2023) [17]	score(w_{ij})=$[\|\mathbf{w}\|^2/\text{diag}((\mathbf{x}^T\mathbf{x}+\lambda I)^{-1})]_{ij}$	U
Wanda (2023) [49]	score(w_{ij})=$\|w_{ij}\|\cdot\|X_j\|_2$	U
LLM-Pruner (2023) [20]	score(w_i)=$\|\nabla_{w_i}\mathcal{L}\odot w_i - \frac{1}{2}\sum_{j=1}^{N}(\nabla_{w_i}\mathcal{L}\odot w_i)^2\|$	S
ShortGPT (2024) [65]	$1-\frac{\mathbf{x}_i^T\mathbf{x}_{i+1}}{\|\mathbf{x}_i\|_2\|\mathbf{x}_{i+1}\|_2}$	S
LLM-Streamline (2024) [109]	$\max_{\mathbf{layer}_i} cos(\mathbf{x}_i,\mathbf{x}_{i+n})$	S

"U/S" denotes unstructured or structured pruning.

propose Dynamic Sparse Reparameterization (**DSR**) which uses an adaptive threshold for pruning. In addition, DSR [95] reallocates weights across layers and does not restrict to the inner-layer weight redistribution in SET [91]. Liu et al. [93] propose an ensemble method **FreeTickets** that ensemble sparse subnetworks created by sparse-to-sparse methods. Rather than using a random regeneration scheme, Liu et al. [92] propose Gradual Pruning with zero-cost Neuroregeneration (**GraNet**) to remove connections of networks based on their weight magnitudes and regrow connections of networks based on their gradient. They argue that even the weights with zero gradient values indicate the connection importance. Sokar et al. [98] pioneer to explore dynamic sparse training in Reinforcement Learning (**RL**).

Evci et al. [99] analyze the reasonability of dynamic sparse training and find that sparse networks have poor gradient flow at initialization, but dynamic sparse training significantly improves gradient flow.

3) Score-Based Methods: Some PDT methods exploit scoring criteria to prune networks during training. He et al. [83] propose Soft Filter Pruning (**SFP**) filter pruning method which can train networks from scratch and prune them simultaneously by using the l_2 norm of each filter as its importance measure. Instead of associating masks with filters, they directly set the pruned filter weights to zero, which can be updated from zero through the forward-backward process. Therefore, pruned filters in one epoch can be recovered in the next. However, SFP requires manually preset a prune ratio for each layer. He et al. [100] propose Filter Pruning via Geometric Median (**FPGM**) to prune redundant filters that are nearest to the geometric median [101] of the filters within the same layer. However, FPGM also requires a pre-defined prune ratio for each layer. Liu et al. [57] propose a method called **Network Slimming** that introduces a scaling factor for each channel and jointly trains the weights and the scaling factors by adding the regular loss ℓ with sparsity regularization on the factors, and the magnitudes of these scaling factors are used as the filter scores. In practice, they reuse the γ parameters in BN layers as the scaling factors.

4) Differentiable Pruning Based Methods: Advances in differentiable network compression ([27], [28]) popularize differentiable techniques for pruning, such as [85], [86], [102], [103], [104]. For example, Ning et al. [85] propose Differentiable Sparsity Allocation (**DSA**) to set layer-wise prune ratios. They soften the hard pruning with a differentiable method using masks drawn from a distribution governed by prune ratios, allowing for gradient calculation of the target loss w.r.t. these ratios. These gradients then measure layer sensitivity. Guo et al. [102] model channel pruning as a differentiable Markov process with architecture parameters, representing each channel as a state and using state transitions to determine the probability of retaining a channel based on the retention of its predecessor. Unlike [85] and [102], Cho et al. [86] generate soft pruning masks for weights without extra trainable parameters to accomplish differentiable pruning for CNNs and Transformers.

C. Pruning After Training

Pruning After Training (**PAT**) (as shown in Table IV) is the most popular type of pruning pipeline because it is commonly believed that pre-training the dense network is necessary to obtain an efficient subnetwork [110]. Especially for large models, such as LLMs and diffusion models, pruning is specifically applied to pre-trained models. This class of pruning methods typically follows a Pretrain-Prune-Retrain ([20], [62]) or Pretrain-Prune ([17], [19]) process, as shown in Fig. 5(c). (1) Pre-train a randomly initialized dense network $f(\mathbf{x};\mathbf{w}_0)$ to converge to $f(\mathbf{x};\mathbf{w}_T)$. (2) Prune the weights (or filters, neurons, etc.) that have the least influence on performance and fine-tune the pruned network $f(\mathbf{x};\mathbf{w}'_T\odot\mathbf{m}')$ for several iterations, where $\mathbf{w}'_T$ and $\mathbf{m}'$ are the weights and masks after pruning, respectively. This step can be done at least once (i.e., one-shot pruning) or multiple times (i.e., iterative pruning). (3) Train the remaining weights from scratch $f(\mathbf{x};\mathbf{w}_0\odot\mathbf{m}'')$ or fine-tune $f(\mathbf{x};\mathbf{w}''_T\odot\mathbf{m}'')$ to recover the performance [111], where $\mathbf{w}''_T$ and $\mathbf{m}''$ are the final results of weights and masks after the pruning process, respectively. Sparsity may gradually increase during the pruning process until it achieves the target.

1) LTH and its Variants: Lottery Ticket Hypothesis (**LTH**) [48], [112] is one of the most influential hypotheses in the neural network pruning domain. Given a pre-trained network, LTH iteratively removes a percentage of the weights based on their magnitudes. After pruning, the remaining weights are retrained from scratch with the original initialization, rather than random reinitialization, to match the original networks' accuracy. It challenges the commonly held belief that pre-trained

weights must be used for retraining and conjectures the existence of an independently trainable sparse subnetwork within a dense network. Inspired by LTH, there are various follow-up works to identify wider tickets across multiple kinds of neural networks (such as CNNs [113], Generative Adversarial Networks (**GANs**) [114], Variational AutoEncoders (**VAEs**) [115], Graph Neural Networks (**GNNs**) [116], Transformer based models [117]) and understand LTH better, which can be classified into five main classes: (1) proposing a stronger lottery ticket hypothesis, (2) exploring the transferability of LTH, (3) generalizing LTH to other contexts, (4) theoretical justification, and (5) revisiting and questioning LTH.

1) Some recent works ([113], [118], [119]) prove stronger hypothesis than LTH [48]. For example, Diffenderfer and Kailkhura [113] propose a stronger Multi-Prize LTH, which claims that winning tickets can be robust to extreme forms of quantization (i.e., binary weights and/or activations). Based on this, they introduce the Multi-Prize Tickets (**MPTs**) algorithm to find MPTs on binary neural networks for the first time.

2) Some literature ([120], [121], [122], [123]) studies the transferability of a winning ticket found in a source dataset to another dataset, which provides insights into the transferability of LTH. For example, S. Morcos et al. [120] find OneTicket that can generalize across a variety of datasets and optimizers within the natural image domain. Mehta [121] propose the ticket transfer hypothesis and transfer winning tickets for different image classification datasets.

3) In addition to image classification, LTH has been extended to other contexts, such as node classification and link prediction ([123]), vision-and-language ([122]), and NLP ([117]). For example, Chen et al. [123] pioneer to generalize LTH to GNNs and propose Graph Lottery Ticket (**GLT**). Prasanna et al. [117] explore the existence of winning tickets for fine-tuned BERT [3] and identify subnetworks that match the model's performance.

4) On one hand, some literature ([124], [125], [126]) analyzes the reasons why LTH [48] is able to win. For example, Zhang et al. [124] exploit dynamical systems theory and inertial manifold to theoretically verify the validity of LTH. Evci et al. [99] observe that the success of LTH lies in effectively re-learning the original pruning solution they are derived. Zhang et al. [125] pioneer to provide a formal justification for the improved generalization of winning tickets observed in experimental results from LTH.

5) On the other hand, some recent works ([110], [127], [128]) revisit and challenge the existence of LTH. For example, Ma et al. [127] provide a more rigorous definition of LTH for precisely identifying winning tickets and find that whether and when the winning tickets can be identified highly replies on the training settings, such as learning rate, training epochs, and architecture characteristics like network capacities and residual connections. It is more likely to find winning tickets by using a small learning rate or an insufficient number of training epochs.

It is worth pointing out that in some works [18], [36], LTH [48] is classified as a PBT method. However, LTH selects masks based on a pre-trained network, which does not conform to the definition of PBT that attempts pruning the initialized network before training. Therefore, it is more reasonable to classify LTH as a PAT method.

2) Other Score-Based Methods: The most straightforward and intuitive way to select pruning candidates is to evaluate them based on their norms. For example, Han et al. [13] propose to measure weight importance by its absolute value. In addition to norm-based criteria, evaluating loss change with and without the weights is also popular. For example, Nonnenmacher et al. [67] propose Second-order Structured Pruning (**SOSP**) to selectively zero out filter masks to minimize the effects of the loss change from removing some filters. Ma et al. [20] propose LLM-Pruner to pioneer the removal of unimportant coupled channels and multi-attention heads in LLMs using the first-order Taylor expansion to estimate loss changes and fine-tune the pruned models using LoRA [129]. Fang et al. [39] present Diff-Pruning which scores weights in diffusion models using the first-order Taylor expansion over pruned timesteps. Shi et al. [130] introduce Unified and Progressive Pruning (**UPop**), which prunes large multimodal models by leveraging accumulated trainable mask gradients during each iteration of the search phase. In addition to using loss change, some works ([65], [131], [132], [133]) design new metrics. For example, Men et al. [65] introduce a metric called Block Influence (**BI**), which assesses the significance of a layer by measuring the extent to which it alters the hidden states and removes redundant layers of LLMs. Kim et al. [133] assess the significance of a layer in LLMs by measuring its impact on perplexity (**PPL**) upon its removal.

3) Sparsity Regularization Based Methods: Some works ([107], [134], [135], [136], [137]) exploit sparsity regularization technique. For example, He et al. [134] propose an alternative two-step algorithm that introduces a scalar mask to each channel of a pre-trained CNN model, selects redundant channels based on LASSO regression [138], and reconstructs the outputs of unpruned channels using linear least squares. Energy-Constrained Compression (**ECC**) [139] builds an energy consumption model via a bilinear regression function. Network Pruning via Performance Maximization (**NPPM**) [140] trains a performance prediction network and uses it as a proxy of accuracy to guide searching for subnetworks based on regularization penalty. Fang et al. [38] develop a general method called **DepGraph** to analyze dependencies in various network structures (e.g., CNNs, RNNs, GNNs, Transformers) and propose a structured pruning based on sparsity regularization. Using the l_0 regularization approach, Xia et al. [137] efficiently learn pruning masks that match the target architecture and maximize performance by jointly optimizing weights and pruning masks with a min-max objective for LLMs. Some methods ([38], [53], [105], [108]) select important weights (or filters, neurons, etc.) by combining norm-based criteria and sparsity regularization.

4) Pruning in Early Training: Instead of fully training a network from $f(\mathbf{x};\mathbf{w}_0)$ to $f(\mathbf{x};\mathbf{w}_T)$, this class of methods explore the network architecture by training a network only for a few iterations or epochs, i.e., $f(\mathbf{x};\mathbf{w}_t)$, where $t << T$. For example, You et al. [10] propose Early-Bird (**EB**) tickets which indicate that winning tickets can be identified at the early training stage via inexpensive training schemes (e.g., early stopping and low-precision training) at large learning rates and achieve similar performance to the dense network. Inspired by [10], Chen et al. [141] propose **EarlyBERT** that identifies structured winning

tickets in the early stage of BERT [142] training. Frankle et al. [143] find that, in large-scale settings (such as ResNet-50 and Inception-v3 on ImageNet), the subnetworks that exhibit stability to SGD noise are able to reach full accuracy early in training.

5) Post-Training Pruning: In contrast to the general PAT methods that follow the Pretrain-Prune-Retrain procedure, recently proposed post-training pruning methods simplify the three-step process Pretrain-Prune. It involves pruning a pretrained model $f(\mathbf{x}; \mathbf{w}_T)$ without retraining, typically achieving negligible accuracy loss by using compensation mechanisms to mitigate performance degradation. This class of pruning methods is particularly attractive for billion-parameter models because retraining such pruned models is still very expensive. For example, Kwon et al. [144] propose a structured post-training pruning framework for Transformers that features Fisher-based mask search, rearrangement, and tuning, achieving pruning on a single GPU in three minutes without retraining. SparseGPT [17], a groundbreaking unstructured post-training pruning method for LLMs, tackles the pruning problem as an approximate sparsity reconstruction problem and prunes LLMs at least 50% sparsity with minor accuracy loss without retraining. To address SparseGPT's reconstruction cost, Wanda [49] uses weight magnitudes and input norms to facilitate unstructured post-training pruning on LLMs without updating weights. Unlike SparseGPT and Wanda, SliceGPT [19] implements structured post-training pruning using orthogonal matrix transformations and principal component analysis (**PCA**) to remove columns and rows of the weight matrices in LLMs. FLAP [145] introduces a fluctuation metric and uses a bias compensation mechanism for LLMs' performance recovery.

D. Run-Time Pruning

The prior works on pruning usually focus on static pruning methods where the pruned model is reused for different inputs. In contrast, some methods prune neural networks according to individual inputs dynamically, known as run-time pruning [146]. This line of work is based on the premise that for a given task, the difficulty of producing accurate output can vary, implying that necessary model capacities for different inputs are different [64]. For example, Rao et al. [146] propose a Runtime Network Routing (**RNR**) framework to conduct dynamic routing based on the input image and current feature maps and select an optimal path subset for compression. Tang et al. [64] point out that the importance of channels highly depends on the input data and propose to generate different subnetworks for each instance. At inference, only channels with saliencies larger than the threshold need to be computed, and the redundant features are skipped. Hua et al. [147] exploit input-specific characteristics and propose **CGNets** to predict unimportant regions by the partial sum of the output activation by performing convolution on a subset of input channels. Gao et al. [148] propose Feature Boosting and Suppression (**FBS**) to predict the saliency of channels and skip those with less contribution to the classification results at run-time. Elkerdawy et al. [149] pose dynamic model pruning as a self-supervised binary classification problem. Meng et al. [150] propose Contrastive Dual Gating (**CDG**), another self-supervised dynamic pruning method that uses contrastive learning [151]. Tuli and Jha [152] introduce DynaTran, which prunes activations at runtime based on the magnitude of the input matrix to enhance transformer inference throughput.

V. Pruning Criteria

In this section, we summarize some commonly used pruning criteria for evaluating the importance of weights (or filters, neurons, etc.) from different perspectives, including magnitude ([12], [153], [154], [155]), norm ([63], [83]), saliency and/or sensitivity ([82], [156]), and loss change ([15], [62], [67], [156], [157]). There is no rigid boundary between these criteria, but a different emphasis.

A. Magnitude-Based Pruning

[31] is one of the earliest works that propose magnitude-based pruning to reduce hidden units. Han et al. [13] popularize magnitude-based pruning for deep neural networks, which prunes the lowest-magnitude weights. It is based on the assumption that weights with smaller absolute values tend to have the least influence on the network's output [63]. The formulation is defined as

$$m_i = \begin{cases} 1 : \text{if } \|w_i\|_1 \geq a \\ 0 : \text{if } \|w_i\|_1 < a \end{cases}, \tag{3}$$

where a is a threshold. Some criteria combine weight magnitude with activation magnitude. For example, the score for a weight w_{ij} in Wanda [49] is defined by

$$s_{ij} = |w_{ij}| \cdot \|\mathbf{x}_j\|_2. \tag{4}$$

In addition to weight and activation, magnitude-based pruning can be applied to other values ([132], [158]). For example, Dery et al. [132] propose to prune LLMs by selecting modules with the highest magnitude of module relevance until the pruning constraint is met.

Magnitude-based criteria can be applied to either unstructured ([48], [49], [153]) or structured pruning ([63], [94], [132]). For example, Li et al. [63] score the filters by calculating the sum of the absolute magnitude of their weights. In addition, magnitude-based criteria can be combined with global/local and one-shot/iterative schedules. For example, the works in [159] and [160] propose magnitude-based iterative global pruning methods. Lubana and Dick [154] argue that magnitude-based pruning results in faster model convergence than magnitude-agnostic methods.

B. l_p Norm

Some methods ([49], [63], [83]) use the l_p norm to evaluate the importance of weights (or filters, neurons, etc.). For example, He et al. [83] exploit the l_p norm to evaluate the importance of the filter $\mathcal{F}_{i,j}$, as shown in (5).

$$\|\mathcal{F}_{i,j}\|_p = \left(\sum_{n=1}^{c_{in}^{(i)}} \sum_{k_1=1}^{k^{(i)}} \sum_{k_2=1}^{k^{(i)}} |\mathcal{F}_{i,j}(n, k_1, k_2)|^p \right)^{\frac{1}{p}}, \tag{5}$$

where $k^{(i)}$ is the kernel size of layer i in a network and $c_{in}^{(i)}$ is the number of channels at layer i. Weights with smaller l_p norms are more likely to be pruned than those with higher l_p norms. In addition, the importance value is often optimized with norm-based sparsity regularization ([105]), which is discussed in Section VI-A.

C. Sensitivity and/or Saliency

Some works ([82], [161], [162]) utilize sensitivity and/or saliency to evaluate the importance of weights (or filters, neurons, etc.). For example, LeCun et al. [161] define weight saliency as the loss change induced by pruning that weight. Lee et al. [54] propose a saliency criterion called the connection sensitivity criterion as the normalized magnitude of the derivatives g_j

$$s_j(\mathbf{w};\mathcal{D}) = \frac{|g_j(\mathbf{w};\mathcal{D})|}{\sum_{k=1}^{m}|g_k(\mathbf{w};\mathcal{D})|}, \tag{6}$$

where s_j is the sensitivity of w_j, $\mathbf{w}$ represents the network's weights, g_j is the derivative of the loss $\mathcal{L}(\mathbf{w}\odot\mathbf{m})$ w.r.t. the mask m_j. The higher the sensitivity, the more important the weight is. Zhao et al. [82] reformulate the BN layer by extending the scale factor γ on the shift term β, which is treated as channel saliency. They reformulate BN as follows:

$$\mathbf{x}_{out} = \gamma \cdot BN(\mathbf{x}) + \tilde{\beta}, \tag{7}$$

where $\tilde{\beta} = \gamma \cdot \beta$. Rather than relying on the value of γ, unimportant channels are pruned based on γ's distributions.

D. Loss Change

It is a widely used criterion to measure the importance of a weight (or filter, neuron, etc.) by evaluating the loss change of the network with and without it. The loss change is usually approximated in a Taylor expansion-based way, such as [15], [20], [39], [62], [67], [163].

The first-order Taylor expansion is the most commonly used method for measuring the loss change. The loss change with a small perturbation at $\mathbf{w}$ is defined as follows:

$$\Delta\mathcal{L} = \mathcal{L}(\mathbf{w}+\Delta\mathbf{w}) - \mathcal{L}(\mathbf{w}) = \nabla_{\mathbf{w}}\mathcal{L}\,\Delta\mathbf{w}. \tag{8}$$

For example, You et al. [15] introduce scaling factors $\boldsymbol{\lambda}$ to the BN and exploits the first-order Taylor expansion to estimate the loss change $\Delta\mathcal{L}$ caused by setting some scaling factors to zero as follows:

$$\Delta\mathcal{L}(\boldsymbol{\lambda}) = |\boldsymbol{\lambda}\nabla_{\boldsymbol{\lambda}}\mathcal{L} - R_1(\boldsymbol{\lambda})| \approx |\boldsymbol{\lambda}\nabla_{\boldsymbol{\lambda}}\mathcal{L}| = \left|\frac{\partial\mathcal{L}}{\partial\boldsymbol{\lambda}}\boldsymbol{\lambda}\right|, \tag{9}$$

where $R_1(\boldsymbol{\lambda})$ is the Lagrange remainder. The importance score of the ith filter is defined as

$$\text{Score}(\mathcal{F}_i) = \sum_{(\mathbf{x},\mathbf{y})\in\mathcal{D}}\left|\frac{\partial\mathcal{L}(\mathbf{y},f(\mathbf{x};\mathbf{w})}{\partial\lambda_i}\lambda_i\right|, \tag{10}$$

where λ_i is the scalar factor of the ith filter.

The second-order Taylor expansion of the loss function is early used in [161], [164] for removing unimportant weights and gradually exploited in many subsequent methods ([62], [67], [163], [165], [166]), which includes the first-order (gradient) term, the second-order (Hessian) term, and the higher-order terms are neglected. Without loss of generality, the approximation of the loss change leads to

$$\mathcal{L}(\mathbf{w}+\Delta\mathbf{w}) - \mathcal{L}(\mathbf{w}) = \nabla_{\mathbf{w}}\mathcal{L}\,\Delta\mathbf{w} + \frac{1}{2}\Delta\mathbf{w}^T H\Delta\mathbf{w}, \tag{11}$$

where $H = \nabla^2_{\mathbf{w}}\mathcal{L}(\mathbf{w})$.

For example, Liu et al. [62] apply the second-order Taylor expansion to approximate the loss change when removing a channel (setting its mask to 0)

$$\begin{aligned} s_i = \Delta\mathcal{L} &= \mathcal{L}(\mathbf{m}-\mathbf{e_i}) - \mathcal{L}(\mathbf{m}) \approx -\mathbf{e_i}^T\nabla_{\mathbf{m}}\mathcal{L} + \frac{1}{2}\mathbf{e_i}^T(\nabla^2_{\mathbf{m}}\mathcal{L})\mathbf{e_i} \\ &= -\mathbf{e_i}^T g + \frac{1}{2}\mathbf{e_i}^T H\mathbf{e_i} = -g_i + \frac{1}{2}H_{ii}, \end{aligned} \tag{12}$$

where $\mathbf{e_i}$ is the one-hot vector with the ith entry equals one, and g is the gradient of the loss function $\mathcal{L}$ w.r.t. $\mathbf{m}$.

VI. Learn to Prune

In this section, we present some methods for learning to prune networks, including sparsity regularization ([57], [59], [134], [167], [168]) and pruning methods based on meta-learning ([84], [167]), graph neural networks ([169]), and reinforcement-learning ([146], [170]).

A. Sparsity Regularization Based Pruning

Sparsity regularization based pruning [132], [134] learns the weights and their masks by solving the following problem:

$$\min_{\mathbf{w},\mathbf{m}} \mathcal{L}(\mathbf{w},\mathbf{m}), \tag{13}$$

where $\mathcal{L} = \ell(\mathbf{w},\mathbf{m}) + \lambda\mathcal{R}(\cdot)$. One common way is to introduce a scaling factor vector $\boldsymbol{\gamma}$ for weights (or channels, filters, etc.). The network weights and the scaling factors $\boldsymbol{\gamma}$ are trained jointly with sparsity regularization imposed on the latter. The magnitude of the scaling factors is treated as the important scores. Specifically, $\mathcal{L}$ in (13) can be exemplified as follows:

$$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\ell(\mathbf{y}_i, f(\mathbf{x}_i;\mathbf{w},\boldsymbol{\gamma})) + \lambda\sum_{\gamma_i\in\boldsymbol{\gamma}}\mathcal{R}(\gamma_i). \tag{14}$$

For example, He et al. [134] cast channel selection as the minimization of reconstruction error in feature maps and formulate the channel pruning problem as follows:

$$\begin{aligned} &\min_{\beta,\mathbf{w}} \frac{1}{2N}\|\mathbf{y} - \sum_{i=1}^{c}\beta_i\mathbf{x}_i w_i^T\|_F^2 + \lambda\|\beta\|_1, \\ &\text{subject to } \|\beta\|_0 \le c', \forall i\ \|w_i\|_F = 1. \end{aligned} \tag{15}$$

where $\|\cdot\|_F$ is the Frobenius norm, $\mathbf{x}_i$ is an $N\times k_h k_w$ matrix from ith channel of input $\mathbf{x}$, w_i is an $n\times k_h k_w$ weight matrix from ith channel of $\mathbf{w}$, k_h and k_w are the kernel height and width, respectively. N, c, c', and n represent the number of samples, channels, retained channels, and output channels. To solve this problem, He et al. [134] use LASSO regression [138] and a greedy strategy to select the unimportant channels.

B. Meta-Learning Based Pruning

Some works ([84], [167]) adopt meta-learning to prune models. For example, Liu et al. [84] train a meta network, PruningNet, to predict weights for different pruned networks. The PruningNet takes a network encoding vector $(v_1, v_2, \ldots, v_L)$ as input and outputs the weights $\mathbf{w}$ of the pruned network

$$\mathbf{w} = \text{PruningNet}(v_1, v_2, \ldots, v_L), \tag{16}$$

where v_i is the number of the channels for the ith layer. The weights and the corresponding accuracy of each pruned network are obtained by inputting the network encoding into the fully trained PruningNet. Considering the huge search space of network encoding vectors, the pruned network is found by evolutionary search under the constraints.

C. Graph Neural Network Based Pruning

Any network can be viewed as a graph. Zhang et al. [169] propose a method called GraphPruning for model compression. Specifically, GraphPruning designs a graph aggregator G with weights $\boldsymbol{\theta}_G$, combined with the Fully Connected (FC) layers, to generate the weights $\mathbf{w} = (\mathbf{w}^{(1)}, \mathbf{w}^{(2)}, \ldots, \mathbf{w}^{(L)})$ of the "Pruned Network" as follows:

$$\begin{aligned}(n_1, n_2, \ldots, n_L) &= G(b_1, b_2, \ldots, b_L|\boldsymbol{\theta}_G),\\ \mathbf{w}^{(i)} &= FC_i(n_i|\boldsymbol{\theta}_i),\end{aligned} \tag{17}$$

where $b_i \in R^{1\times 7}$ denotes the embedding features of the ith node, n_i is the ith column of the output with the graph aggregator, θ_i are the weights of the ith FC layer, and $\mathbf{w}^{(i)}$ are the weights of the ith pruned layer of the pruned network. Then, the "Pruned Network" is fully trained. The graph aggregator is responsible for extracting high-level features for each node, while each FC layer is used to generate reasonable weights for the "Pruned Network". Afterward, the best configuration of the "Pruned Network" under computational constraints is searched by RL methods, during which the weights of the graph aggregator and FCs are not updated.

D. Reinforcement Learning Based Pruning

Rather than using RL to search for the best configurations of the pruned networks as in [169], some AutoML pruning methods ([146], [170]) adopt RL to compress models automatically. For example, He et al. [170] propose AutoML for Model Compression (**AMC**), which is based on Q-learning, a type of RL that focuses on how an agent should take actions to maximize the cumulative reward. Specifically, He et al. [170] design the Deep Deterministic Policy Gradient (**DDPG**) agent to receive an embedding state s_i of layer l_i from the environment and output a sparsity ratio as action a_i. Then, layer l_i is compressed with a_i using a specific compression method (such as a channel pruning method). After that, the agent moves to layer l_{i+1} and repeats the same process until the final layer l_L. The update process is as follows:

$$\begin{aligned}Loss &= \frac{1}{N}\sum_{i=1}^{N}(\mathbf{y}_i - Q(s_i, a_i|\mathbf{w}^Q))^2,\\ \mathbf{y}_i &= r_i - b + \gamma Q(s_{i+1}, \mu\,(s_{i+1})|\mathbf{w}^Q),\end{aligned} \tag{18}$$

where b is the baseline reward, γ is a discount factor used to avoid over-prioritizing short-term rewards, $\mathbf{w}^Q$ are the weights of the network Q following Block-QNN [171], and r_i is the reward of the whole trajectory for the ith sample.

He et al. [170] observe that Error is inversely-proportional to log(FLOPs) or log(#Param). Based on this observation, the reward function is defined as

$$\begin{aligned}\text{R}_{\text{FLOPs}} &= -\text{Error} \cdot log(\text{FLOPs}),\\ \text{R}_{\text{Param}} &= -\text{Error} \cdot log(\#\text{Param}).\end{aligned} \tag{19}$$

This reward function provides an incentive for reducing FLOPs or the number of network parameters.

VII. A Comprehensive Comparative Analysis

In this section, we compare some pruning methods on commonly used models, including eight pairs of contrast settings for pruning, different layer-wise densities, and various supervision levels for pruning. To avoid the influence of specific functions on pruning results, we mainly use the same functions under contrast settings (experimental details in Appendix B), available online. Appendix C, available online, provides a more extensive comparison across different methods.

A. Unstructured versus Structured Pruning

Unstructured pruning methods ([13], [17]) remove weights anywhere and can achieve high prune ratios with little impact on accuracy. In contrast, structured pruning ([20], [62], [67]) conducts pruning on entire filters (or channels, neurons, layers, etc.), resulting in really compressed network and accelerated inference. However, the accuracy is often lower than that of unstructured pruning under the same prune ratio, weight-level scoring, pipeline, and learning schemes. The possible reason is that unstructured pruning only focuses on the importance of individual weights, while structured pruning forces structural coupling, which demands simultaneous pruning across multiple layers and expects all removed weights to be consistently unimportant. However, achieving consistency in identifying unimportant weights under the structural coupling constraints is challenging. Amersfoort et al. [172] argue that SNIP-structured and GraSP-structured methods incur more noise than their vanilla unstructured counterparts.

We compare unstructured and structured pruning methods on VGG-16 [1] and report the best results in Table V from three random runs. Additionally, we compare unstructured, semi-structured, and structured pruning methods on OPTs [174] with data sourced from [163] (Table VI). As shown in Tables V and VI, at the same prune ratio, unstructured pruning generally outperforms semi-structured (if any), which performs better than structured pruning.

TABLE V
TOP-1 ACCURACY (%) OF UNSTRUCTURED AND STRUCTURED PRUNING ON VGG-16

Dataset	CIFAR-10		CIFAR-100	
Method \Ratio (%)	10.00	40.00	10.00	50.00
SNIP-unstructured [54]	**93.84**	**93.73**	**73.09**	**72.67**
SNIP-structured	93.74	93.71	72.82	70.68
GraSP-unstructured [56]	93.58	**93.18**	**72.46**	**71.42**
GraSP-structured	**93.67**	93.04	72.43	71.30

"Ratio" refers to the percentage of parameters removed from the original count. **Bold**/underline marks the best/second best performance, respectively, among the compared entities. Unless otherwise specified, "ratio" and **bold**/underline in other tables have the same meaning.

TABLE VI
PERPLEXITY OF UNSTRUCTURED, SEMI-STRUCTURED, AND STRUCTURED PRUNING ON LLMS WITH WIKITEXT2 [173], WHERE LOWER IS BETTER

Method	Ratio (%)	Model			
		O-125m	O-1.3B	O-2.7B	O-6.7B
Unpruned	0	27.65	14.62	12.47	10.86
SparseGPT-unstruct [17]	50.00	**33.17**	**26.77**	**12.88**	**11.92**
SparseGPT-2:4 [17]		45.51	29.44	14.92	13.01
K-OBD-2:4 [163]	50.00	**68.74**	**27.22**	**20.23**	**15.55**
K-OBD-struct [163]		75.95	37.68	26.88	25.54
LLM-Surgeon-unstruct [163]	50.00	**30.30**	**15.47**	**12.68**	**10.97**
LLM-Surgeon-2:4 [163]		44.64	25.10	14.64	12.10
LLM-Surgeon-struct [163]		49.78	22.95	17.15	14.90

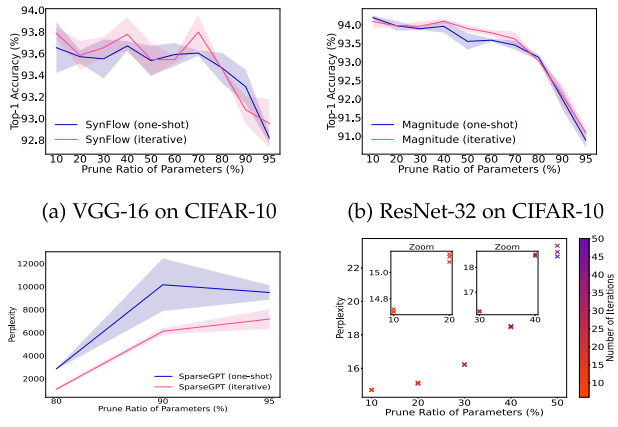
"O" denotes OPT [174].

B. One-Shot versus Iterative Pruning

One-shot pruning methods score once and then prune the network to a target prune ratio. Conversely, the iterative pruning methods alternately process the score-prune-update cycle until achieving the target prune ratio. As a result, the pruning cost in one-shot methods is usually negligible, greatly saving pruning efforts. However, these methods are not beneficial to those significant weights whose importance is not immediately apparent at the beginning [120]. Therefore, one-shot pruning generally requires more carefully designed scoring criteria to match the performance of the original network. In addition, the results in [50] show that one-shot pruning may more easily suffer from layer collapse, resulting in a sharp accuracy drop. In contrast, iterative methods require more pruning cost but generally yield better accuracy [15], [63], [121], [128], [175].

Lin et al. [96] analyze the difference between one-shot and iterative pruning methods from the perspective of stochastic gradient. Their results show that the iterative pruning method computes a stochastic gradient at the pruned model and takes a step that best suits the compressed model. In contrast, one-shot pruning computes a stochastic gradient at the original weights and moves towards the best dense model. The work in [163] theoretically supports iterative pruning by arguing that the surrogate loss landscape, based on a Taylor expansion, only holds locally, making it unreliable for larger weight changes.

We prune VGG-16 [1] and ResNet-32 [9] on CIFAR-10 [176] using SynFlow [50] and Magnitude-based pruning, and LLaMA-7B [177] on PTB [178] using SparseGPT [17], applying one-shot and iterative pruning, respectively. Additionally, we compare the pruning results of OPT-1.3B [174] on WikiText2 [173] under different iteration settings, using experimental results from [163]. As illustrated in Fig. 7(a)–(c), iterative pruning generally performs better than that of the corresponding one-shot pruning and Fig. 7(d) indicates that more iterations tend to yield better performance.



(a) VGG-16 on CIFAR-10 (b) ResNet-32 on CIFAR-10

(c) LLaMA-7B on PTB (d) OPT-1.3B on WikiText2

Fig. 7. One-shot versus iterative pruning. Shaded regions indicate standard deviation based on three independent runs. Best view in color and zoom in.

TABLE VII
TOP-1 ACCURACY (%) OF DATA-FREE (FIRST THREE) AND DATA-DRIVEN (LAST TWO) PRUNING METHODS

Model	VGG-16*		ResNet-32*		ResNet-50	
Dataset	CIFAR-10		CIFAR-100		ImageNet	
Method \Ratio (%)	50.00	90.00	50.00	90.00	73.80	89.30
Random	93.12	90.68	72.10	67.29	71.20	65.20
Magnitude	93.32	92.78	71.86	**68.77**	72.50	66.50
SynFlow [50]	**93.74**	93.26	**72.15**	68.44	72.60	**68.00**
SNIP [54]	93.70	**93.40**	70.87	68.00	**72.70**	66.60
GraSP [56]	92.97	92.51	72.13	68.56	72.10	67.20

Original TOP-1 accuracy on CIFAR-10, CIFAR-100, and ImageNet are 93.76%, 73.18%, and 76.20%, respectively. An asterisk (*) denotes a result from our reproduction; others are from [76].

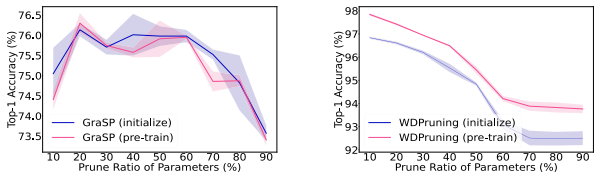
C. Data-Free versus Data-Driven Pruning

Pruning methods can be categorized into two types based on whether data is used during the pruning phase: data-free and data-driven. Data is generally believed to be essential for finding good subnetworks. Most existing pruning works ([20], [54], [62], [67], [130]) belong to data-driven methods, and only a few methods ([50], [55], [179]) are data-free. We apply PBT methods, including three data-free (Random, Magnitude, and SynFlow [50]) and two data-driven (SNIP [54] and GraSP [56]) methods, to prune VGG-16 and ResNet-32/50 on CIFAR-10/100 and ImageNet [180], respectively. Table VII shows that SynFlow and SNIP are similarly effective, with SynFlow significantly outperforming GraSP, indicating that the effectiveness of PBT methods is not strictly dependent on data usage. In addition, we utilize PAT methods, selecting Random and L1/L2-norm as data-free methods, and data-driven approaches such as Wanda [49], LLM-Pruner [20], and LoRAPruner [181] to prune LLaMA-7B. In contrast, Table VIII consistently demonstrates that data-driven PAT methods typically outperform data-free PAT methods.

TABLE VIII
ZERO-SHOT ACCURACY (%) OF DATA-FREE (FIRST THREE) AND DATA-DRIVEN (LAST THREE) PRUNING METHODS ON LLAMA-7B [177] WITH COMMON SENSE DATASETS

Ratio (%)	Method	BoolQ	PIQA	HellaSwag	WinoGrande	Avg. ↑
0	Unpruned	73.18	78.35	72.99	67.01	72.88
20.00 w/ tune	Random*	55.57	73.39	64.49	60.38	63.46
	L1-norm*	58.47	75.35	65.40	60.93	65.04
	L2-norm*	65.02	75.14	65.07	62.12	66.84
	Wanda [49]	**65.75**	74.70	64.52	59.35	66.08
	LLM-Pruner [20]	64.62	77.20	68.80	**63.14**	68.44
	LoRAPruner [181]	65.62	**79.31**	**70.00**	62.76	**69.42**
50.00 w/ tune	Random*	61.04	62.30	40.37	53.51	54.31
	L1-norm*	40.73	66.32	42.66	51.85	50.39
	L2-norm*	38.50	67.08	43.47	52.80	50.46
	Wanda [49]	50.90	57.38	38.12	**55.98**	50.60
	LLM-Pruner [20]	60.28	69.31	47.06	53.43	57.52
	LoRAPruner [181]	**61.88**	**71.53**	**47.86**	55.01	**59.07**

"Avg." is calculated among four datasets. An asterisk (*) denotes a result obtained from our reproduction; others are from [181].



(a) ResNet-152 on CIFAR-100 (b) DeiT-Tiny on CIFAR-10

Fig. 8. Pruning using randomly initialized versus pre-trained weights. Shaded regions indicate standard deviation based on three independent runs (best view in color).

D. Pruning on Initialized versus Pre-Trained Weights

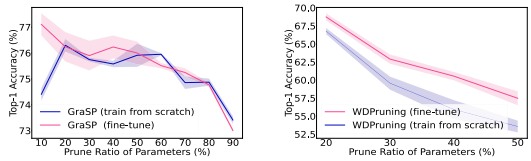
Frankle et al. [76] find the subnetworks in CNNs obtained by pruning on randomly initialized weights (such as SNIP [54], GraSP [56], SynFlow [50]) are robust to ablation treatments (i.e., randomly shuffling the mask positions within each layer or reinitializing weights while keeping masks unchanged). To find the reason behind such immunity, Singh and Liu [182] use the Wasserstein distance to measure distributional similarity and find that the remaining weights' distribution changes minimally with these ablations, which helps maintain similar performances. In contrast, Su et al. [55] find the subnetworks in CNNs achieved by pruning on pre-trained weights, such as LTH [48], are sensitive to these ablations. Qiu and Suda [183] claim that training weights can be decoupled into two dimensions: the locations of weights and their exact values, with the locations of weights holding most of the information encoded by the training. Wolfe et al. [184] theoretically analyze the impact of pre-training on the performance of a pruned subnetwork in CNNs obtained by greedy forward selection and find that the number of pre-training iterations increases logarithmically with the dataset size. Unlike CNNs, Transformers typically need appropriate self-supervised pre-training to perform well [185]. Therefore, pruning of Transformers generally targets pre-trained models. We conduct experiments to explore whether pre-trained weights facilitate achieving better subnetworks. Fig. 8(a) indicates that for the PBT method, GraSP [56], pruning with pre-trained weights does not necessarily improve Top-1 accuracy. However, Fig. 8(b) shows that for the PAT method, WDPruning [158], pre-training may be crucial for obtaining subnetworks with better performance.

E. Global versus Local Pruning

The difference between global ([62], [163], [170], [186], [187]) and local ([16], [20], [53], [105], [108]) pruning lies in whether structures are removed from a subset or all available structures of a network. A major limitation of local pruning is that setting a pre-defined prune ratio for each layer can be complex and lead to sub-optimal sparsity. To simplify, local pruning often uses a consistent prune ratio across layers. In contrast, global pruning automatically generates a varying prune ratio for each layer. However, global pruning poses great challenges, particularly for LLMs, due to significant variations in layer magnitudes. For instance, some outlier features may have magnitudes up to 20 times larger than others [188], leading to incomparability issues. Ma et al. [20] notes a marginal advantage of local pruning compared to global pruning in LLMs. Although previous pruning methods for LLMs ([17], [20], [49], [181]) are primarily local, some global methods ([145], [163], [189]) start to emerge. For example, Bai et al. [189] argue that local pruning excessively restricts the alignment of input and output in all the intermediate layers, leading to a sub-optimal solution, and propose a global pruning method called SparseLLM to address the drawbacks.

F. Training From Scratch versus Fine-Tuning

After pruning, many pruning methods require training the subnetwork for several epochs to regain performance. Le and Hua [190] argue that retraining is essential to recover loss accuracy in pruning. Generally, retraining can be divided into two types: training from scratch or fine-tuning. There has been debate over whether fine-tuning is more effective than training from scratch in recovering accuracy. On the one hand, Liu et al. [110] find that for ResNet, VGG, and other standard structures on ImageNet, training the subnetworks with new random initialization can achieve better performance than fine-tuning them. On the other hand, Li et al. [63] observe that training a subnetwork from scratch performs worse than fine-tuning it. Liu et al. [128] investigate pruning ResNet20 on CIFAR-10 with ADMM-based [191] one-shot pruning method and find that pruning & fine-tuning outperforms LTH (pruning & training from scratch) over various prune ratios. Additionally, the results in [140], [192] show that fine-tuning is necessary for better performance on sparse mobile networks than training from scratch. The results in [39] reveal that training from scratch requires more steps to achieve convergence, suggesting that starting pruned models from scratch may not be the most cost-effective strategy, given its training cost is comparable to that of pre-trained models. In recent years, some compromise methods (such as weight rewinding [143]) have been proposed. The results in [111], [143] show that weight rewinding can achieve higher accuracy than fine-tuning. We prune ResNet-152 and DeiT-Tiny [193] on CIFAR-100 and ImageNet with pre-trained weights, respectively. Then we fine-tune the pruned networks or train them from scratch. Fig. 9 indicates that fine-tuning generally outperforms training from scratch. Notably, on ImageNet, fine-tuning achieves significantly higher accuracy than training from scratch.



(a) ResNet-152 on CIFAR-100 (b) DeiT-Tiny on ImageNet

Fig. 9. Training from scratch versus fine-tuning. Shaded regions indicate standard deviation based on three independent runs (best view in color).

G. Original Task versus Transfer Pruning

In recent literature, pruning is combined with transfer learning [194] that can dramatically improve accuracy and speed up convergence. For ease of distinction, in this survey, original task pruning denotes the pruning pipeline that directly performs on the target task. In contrast, transfer pruning is performed on the source task and then transfers the subnetwork to the target task. Specifically, transfer pruning is divided into two types: dataset transfer and architecture transfer. The former prunes networks on the source dataset and transfers the subnetwork to the target dataset, while the latter prunes on one architecture and transfers the subnetwork to another.

Some works ([121], [187], [195], [196]) study the transferability of sparsity masks on datasets. S. Morcos et al. [120] observe that for image classification, winning tickets generated on larger datasets (such as with larger training set size and/or more classes) consistently transfer better than those generated with smaller datasets. For example, winning tickets generated on ImageNet and Places365 demonstrate better performance across other smaller target datasets such as CIFAR-10 and CIFAR-100. Iofinova et al. [197] present a pioneering study of the transfer performance of subnetworks and find that pruning methods with similar Top-1 accuracy on ImageNet [180] can have surprisingly different Top-1 accuracy when used for transfer learning. For pruning on LLMs, performing pruning directly on the target task typically yields better performance than zero-shot pruning [163]. For architecture transfer pruning, Elastic Ticket Transformations (**ETTs**) [198] transforms winning tickets in one kind of network (such as ResNet [9]) to another deeper or shallower one from the same model family.

H. Static versus Dynamic Pruning

Static pruning [14] uses static pruning criteria and removes components. In contrast, dynamic pruning ([64], [147], [148], [199]) exploits input-specific pruning criteria, preserves the entire network structures, and accelerates the networks by dynamically skipping unimportant components. However, dynamic pruning generally does not perform run-time fine-tuning or retraining. The difference between static and dynamic pruning is mainly reflected in the pruning criteria and the pruned model. The advantages and disadvantages of static and dynamic pruning are shown in Table IX.

I. Layer-Wise Weight Density Analysis

Some works ([62], [127], [169], [200]) study the distributions of layer-wise weight density in a subnetwork, showing that different layers can have very different weight densities. The differences arise from the joint action of networks' structural characteristics and pruning methods. Zhang et al. [201] empirically divide the layers into either "ambient" or "critical". Ambient layers are not sensitive to weight changes, while critical layers are. Thus, ambient layers should be heavily pruned, resulting in lower weight densities.

Pruning methods can also result in different weight densities. Gong et al. [202] categorize the sparsity allocation formed by pruning methods into uniform and non-uniform. Uniform sparsity methods ([17], [20], [49]) allocate the same prune ratio for all the layers, whereas non-uniform methods ([145], [163], [189], [203]) assign varying sparsity rates to different layers. For example, in CNNs, some pruning methods tend to assign more weights to earlier layers than to later ones. Ma et al. [127] investigate the layer-wise keep-ratios of subnetworks obtained by GraSP [56], SNIP [54], and LTH [48] on VGG and ResNet, observing a common trend of declining layer-wise keep-ratios, except for some special layers (such as the downsampling layers in ResNet). In contrast, Liu et al. [62] find their pruned networks keep higher percentages of channels in the deeper layers than those in the lower layers for image classification. For pruning Transformers, the results in [200] show that global magnitude pruning tends to prune Transformer layers uniformly, while global first-order methods heavily prune the deeper layers. Yang et al. [60] discover a unique less-more-less distribution among stacked ViT blocks. LLM surgeon [163] prunes relatively more in the first layers and less in the middle layers. Some works ([65], [133], [204], [205]) reveal that some layers are not essential and can be entirely removed or merged.

TABLE IX
ADVANTAGES AND DISADVANTAGES OF STATIC AND DYNAMIC PRUNING

Type	Advantages	Disadvantages
Static Pruning	model size reduced	fixed subnetwork
Dynamic Pruning	flexible subnetwork	model size unreduced

J. Pruning With Different Levels of Supervision

In descending order of supervision level during neural network pruning, pruning can be divided into supervised, semi-supervised, self-supervised, and unsupervised pruning [206]. Self-supervised learning can be divided into two classes: generative and contrastive learning [207]. Similar to supervised learning, supervised pruning works on fully labeled datasets, and most current pruning methods fall into this category. However, supervised pruning suffers from similar bottlenecks as supervised learning, such as the expensive manual labeling. As a promising alternative, semi-supervised, self-supervised, and unsupervised pruning have drawn massive attention.[4]

For example, Caron et al. [208] observe different results in self-supervision pruning compared to supervision pruning, where winning tickets initialization only introduces a slight performance improvement compared to random re-initialization.

[4]The differences between supervised, semi-supervised, unsupervised, self-supervised learning refer to [207].

Pan et al. [209] claim that unsupervised pruning usually fails to preserve the accuracy of the original model. Notably, label supervision for network pruning and training can be independent. For example, Chen et al. [196] use supervised pruning method **IMP** (Iterative Magnitude Pruning) to explore the subnetworks of self-supervised pre-trained models (simCLR [210] and MoCo [211]) on ImageNet. Similarly, Jeff Lai et al. [212] exploit the supervised pruning method IMP to prune self-supervised speech recognition models.

VIII. Fusion of Pruning and Other Compression Techniques

In this section, we review the fusion of neural network pruning with other network compression techniques, such as quantization [20], tensor decomposition [157], knowledge distillation [213], and network architecture search [68]. On the one hand, fusion provides more choices for network compression. On the other hand, combined compression techniques can complement each other to further improve the performance and prune ratio.

Pruning & Quantization: Quantization [214] is a compression technique that reduces the number of bits used to represent the network weights and/or activations, significantly reducing the model size and memory footprint with only a minor performance drop. To obtain more compact models and achieve model acceleration, Han et al. [13] pioneer pruning the redundant network connections and quantizing the weights. CLIP-Q [214] jointly performs pruning and quantization during the fine-tuning stage. MPTs [113] integrates pruning and quantizing randomly weighted full-precision neural networks to obtain binary weights and/or activations. EB [10] applies 8-bit low-precision training to the stage of searching EB tickets.

Pruning & Tensor Decomposition: Tensor decomposition [22] decomposes convolutions into a sequence of tensors with fewer parameters. In contrast to pruning, it explores the original weights' low-rank structure, while keeping the dimension of the convolutional output unchanged. CC [157] combines channel pruning and tensor decomposition to compress CNN models by simultaneously learning model sparsity and low rankness. Hinge [215] introduces group sparsity to fuse filter pruning and decomposition under the same formulation. Li et al. [216] propose LoSparse to prune Transformers by combining low-rank approximations and pruning.

Pruning & NAS: Neural Architecture Search (**NAS**) provides a mechanism to automatically discover the best architecture for the problem of interest, offering a new approach for pruning to find suitable network depth and width. For example, for CNNs, NPAS [68] performs a compiler-aware joint network pruning and NAS, determining the filter type (different kernel sizes), the pruning scheme, and the rate for each layer. TAS [217] exploits NAS to search for the depth and width of a network to obtain pruned networks and uses knowledge distillation to train these pruned networks. Klein et al. [218] explore structured pruning of fine-tuned Transformers via NAS.

Pruning & Knowledge Distillation: Knowledge Distillation (**KD**) [213] guides the student to effectively inherit knowledge from the teacher and mimic the teacher's output. Some works ([219], [220]) exploit pruning before KD to boost KD's quality. For example, Liu et al. [219] prune unimportant channels to the contents of interest and focus the distillation on the interest regions. Park and No [220] prune the teacher network first to make it more transferable and then distill it to the student. Some works ([147], [221], [222], [223]) use KD to train the pruned networks. The results in [221] show that the pruned network recovered by KD performs better than it regained by fine-tuning. Zou et al. [224] propose a data-free deraining model compression method that distills the pruned model to fit the pre-trained model. [225] introduce a multi-stage compression strategy, AntGMM, to compress large multimodal models by utilizing structured pruning and knowledge distillation.

Pruning & Multi-compression Techniques: Some works ([226], [227], [228]) explore the fusion of pruning with more than one compression technique. For example, GS [228] combines pruning, quantization, and KD for GANs compression. Joint-DetNAS [227] integrates pruning, NAS, and KD for image translation. LadaBERT [226] merges pruning, matrix factorization, and KD to compress BERTs [142] for natural language understanding.

IX. Suggestions and Future Directions

In this section, we discuss how to choose different pruning methods and provide promising directions for future work.

A. Recommendations on Pruning Method Selection

After years of research and exploration, there are many off-the-shelf pruning methods. However, no golden standard exists to determine which one is the best. Different suitable pruning methods exist to compact deep neural networks for specific application requirements and hardware/software resources. Here are some general recommendations for choosing an appropriate pruning method.

1) If you do not have special hardware (e.g., FPGAs or ASICs) or software (such as sparsity convolutional libraries) but need actual neural network acceleration and compression, structured pruning is more suitable than unstructured pruning because most software frameworks and hardware cannot accelerate sparse matrices' computation.

2) If you have sufficient computational resources during the pruning stage, consider using iterative PAT methods that can typically minimize the impact on performance under the same prune ratio. On the other hand, if you have limited computational resources during both the pruning and inference stages, consider using one-shot PBT or one-shot post-training pruning methods, particularly for LLMs.

3) If you have enough labeled examples on the target task, consider using supervised pruning methods. However, if only a few examples on the target task are labeled, semi-supervised or transfer pruning methods may be considered. If the examples on the target task are not labeled, consider self-supervised, unsupervised, or transfer pruning methods.

4) If you have a sufficient memory footprint during pruning for NLP tasks, consider heavily compressing large models rather than lightly compressing smaller models to meet the same budgets. Some results in [229] show that for NLP tasks finding

pruned models derived from larger dense networks outperform small dense networks of comparable size to pruned models.

5) If you have enough memory to store the dense neural network during the inference stage and wish to provide run-time flexible computational cost allocation for different inputs, dynamic pruning methods can be considered where inputs with smaller shapes can allocate less computational cost to perform the task and if the input shape is bigger more computational cost can be allocated.

6) If you need to trim down neural networks in multiple dimensions, you can comprehensively consider layerwise pruning (decreasing the model's depth), channel pruning (reducing the model's width), and image resolution pruning (scaling down the model's input resolution) or token pruning (selectively removing tokens from text data). In addition, pruning can be integrated with quantization to further reduce the memory footprint and the neural networks' size.

7) If you want to achieve a better tradeoff between speed and accuracy, the following settings may help: use a pre-trained model; set an appropriate learning rate (if any) for both the pruning and retraining stages; fine-tune the pruned models for several epochs; integrate pruning with knowledge distillation, NAS, or other compression methods to achieve complementarity; and adversarial training may have some help [122].

8) If you need to train a subnetwork to recover performance, Ma et al. [127] show that subnetworks with residual connections achieve higher accuracy using a relatively small learning rate. In contrast, subnetworks without residual connections benefit from a larger learning rate.

9) To benefit from large-scale pre-training, adding parameters is more critical than minimizing FLOPs [230]. By incorporating techniques such as dynamic convolution [231], models with low FLOPs can increase their capacity without a substantial rise in computational cost, enhancing their performance during extensive pre-training.

B. Future Directions

We discuss four promising directions for the further development of neural network pruning, namely, (1) theories, (2) techniques, (3) applications, and (4) evaluation.

Theories: Despite the existing works, several fundamental questions about pruning still need to be answered. For example, prior works demonstrate that network layers contain irreplaceable information as long as redundant ones. Does a theoretical upper bound of the prune ratio exist for a given network that still maintains the performance of its dense equivalent? In other words, how heavily can a network be pruned theoretically without accuracy loss? It is a tricky question due to the intricate relationships between network layers. Besides, is pruning explainable? A common belief is that deep neural networks are hard to interpret. As such, making pruning explainable is an uphill task. However, the interpretability of pruning is vital for understanding the factors behind pruning (e.g., model structure and weights) and exploring more effective pruning methods.

Techniques: To obtain better algorithm designs whose architectures are learned in an economical, efficient, and effective manner, it is a trend to extend Automated Machine Learning (**AutoML**) methods and NAS to pruning. Furthermore, pruning is also beginning to combine with various learning contexts, such as lifelong learning [222], continual learning [232], contrast learning [233], and federated learning [234], etc. In addition, the rising energy consumption of networks requires more attention to energy-aware pruning. However, preliminary efforts mainly focus on reducing computation and memory costs, which may not necessarily reduce the most energy consumption. Moreover, incorporating pruning into hardware to help deploy pruned networks is also an emerging trend. For example, Sui et al. [235] propose a hardware-friendly pruning method and deploy the pruned models on an FPGA platform.

Applications: Pruning has begun to draw attention to more complex applications such as visual question answering, natural language understanding, speech recognition, and content generation than image classification. Foundation models such as GPT-4 [236] might be a possible way to Artificial General Intelligence (**AGI**). However, its enormous size hinders its application in many downstream tasks. Fig. 2 highlights content related to large model pruning. In the future, more pruning methods will enable colossal foundation models to benefit from pruning research, making them more compact and efficient [17].

Evaluation: With the emergence of many pruning methods, standardized benchmarks, and metrics are required to provide a fair evaluation. Different pruning techniques, network architectures, tasks, and experimental settings lead to incomparable results and make it hard to compare pruning methods fairly [237]. ShrinkBench [42] takes the first step and provides a benchmark of pruning methods for image classification. As pruning is applied to applications beyond image classification, standardized benchmarks and metrics for other applications are needed.

X. Conclusion

As an essential compression technique, deep neural network pruning has attracted increasing research attention with the recent emergence of various pruning methods and applications. This survey conducts a comprehensive review on the following four scopes: 1) universal/specific speedup, with a systematic review of unstructured, structured, and semi-structured pruning; 2) when to prune, including pruning before/during/after training for static pruning and run-time pruning; 3) how to prune, including pruning by criteria and by learning; 4) fusion of pruning with other compression techniques, such as KD and NAS. A comprehensive comparative analysis, including eight pairs of contrast settings for pruning, layer-wise weight density, and different supervision levels, can help researchers to efficiently and effectively grasp the characteristics of different pruning methods. In addition, recommendations on pruning method selection and future research directions are highlighted and discussed. To facilitate future research, real-world miscellaneous applications and commonly used resources of datasets, networks, and evaluation metrics in different applications are summarized in Appendix D, available online. To help researchers and practitioners keep up with the development of pruning technologies, we continue updating the representative research efforts and open-source codes for pruning at https://github.com/hrcheng1066/awesome-pruning.

REFERENCES

[1] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. Int. Conf. Learn. Representations*, 2015, pp. 1–14.

[2] A. Dosovitskiy et al., "An image is worth 16x16 words: Transformers for image recognition at scale," in *Proc. Int. Conf. Learn. Representations*, 2021.

[3] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. Annu. Conf. North Amer. Chapter Assoc. Comput. Linguistics*, 2019, pp. 4171–4186.

[4] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," 2022, *arXiv:2204.02311*.

[5] T. Bohnstingl, A. Garg, S. Woźniak, G. Saon, E. Eleftheriou, and A. Pantazi, "Towards efficient end-to-end speech recognition with biologically-inspired neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021.

[6] S. Latif et al., "Sparks of large audio models: A survey and outlook," 2023, *arXiv:2308.12792*.

[7] J. Lin et al., "VILA: On pre-training for visual language models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 26689–26699.

[8] Y. Liu et al., "Sora: A review on background, technology, limitations, and opportunities of large vision models," 2024, *arXiv:2402.17177*.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.

[10] H. You et al., "Drawing early-bird tickets: Towards more efficient training of deep networks," in *Proc. Int. Conf. Learn. Representations*, 2020.

[11] J. Wu, W. Gan, Z. Chen, S. Wan, and H. Lin, "AI-generated content AIGC: A survey," 2023, *arXiv:2303.04226*.

[12] S. Han, J. Pool, J. Tran, and W. Dally, "Learning both weights and connections for efficient neural network," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2015, pp. 1135–1143.

[13] S. Han, H. Mao, and W. J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding," in *Proc. Int. Conf. Learn. Representations*, 2016.

[14] X. Dong, J. Huang, Y. Yang, and S. Yan, "More is Less: A more complicated network with less inference complexity," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 1895–1903.

[15] Z. You, K. Yan, J. Ye, M. Ma, and P. Wang, "Gate Decorator: Global filter pruning method for accelerating deep convolutional neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 2130–2141.

[16] J.-H. Luo, J. Wu, and W. Lin, "ThiNet: A filter level pruning method for deep neural network compression," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 5068–5076.

[17] E. Frantar and D. Alistarh, "SparseGPT: Massive language models can be accurately pruned in one-shot," 2023, *arXiv:2301.00774*.

[18] V. Sehwag, S. Wang, P. Mittal, and S. Jana, "HYDRA: Pruning adversarially robust neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1649.

[19] S. Ashkboos et al., "SliceGPT: Compress large language models by deleting rows and columns," in *Proc. Int. Conf. Learn. Representations*, 2024.

[20] X. Ma, G. Fang, and X. Wang, "LLM-Pruner: On the structural pruning of large language models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 21702–21720.

[21] E. Denton, W. Zaremba, J. Bruna, Y. LeCun, and R. Fergus, "Exploiting linear structure within convolutional networks for efficient evaluation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2014, pp. 1269–1277.

[22] S. Lin, R. Ji, C. Chen, D. Tao, and J. Luo, "Holistic CNN compression via low-rank decomposition with knowledge transfer," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 12, pp. 2889–2905, Dec. 2019.

[23] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient finetuning of quantized LLMs," 2023, *arXiv:2305.14314*.

[24] W. Shao et al., "OmniQuant: Omnidirectionally calibrated quantization for large language models," in *Proc. Int. Conf. Learn. Representations*, 2024.

[25] Y. Gu, L. Dong, F. Wei, and M. Huang, "MiniLLM: Knowledge distillation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2024.

[26] X. Xu et al., "A survey on knowledge distillation of large language models," 2024, *arXiv:2402.13116*.

[27] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Representations*, 2019.

[28] M. Zhang, S. Su, S. Pan, X. Chang, E. Abbasnejad, and R. Haffari, "iDARTS: Differentiable architecture search with stochastic implicit gradients," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12557–12566.

[29] X. Ding, X. Zhang, N. Ma, J. Han, G. Ding, and J. Sun, "RepVGG: Making VGG-style ConvNets great again," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 13733–13742.

[30] X. Chu et al., "MobileVLM V2: Faster and stronger baseline for vision language model," 2024, *arXiv:2402.03766*.

[31] S. Hanson and L. Pratt, "Comparing biases for minimal network construction with back-propagation," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1988, pp. 177–185.

[32] R. Mishra, H. Prabhat Gupta, and T. Dutta, "A survey on deep neural network compression: Challenges, overview, and solutions," 2020, *arXiv: 2010.03954*.

[33] D. Ghimire, D. Kil, and S. Heum Kim, "A survey on efficient convolutional neural networks and hardware acceleration," *Electronics*, vol. 11, no. 6, p. 945, 2022.

[34] S. Park, J. Choi, S. Lee, and U. Kang, "A comprehensive survey of compression algorithms for language models," 2024, *arXiv:2401.15347*.

[35] M. Xu et al., "A survey of resource-efficient LLM and multimodal foundation models," 2024, *arXiv:2401.08092*.

[36] H. Wang, C. Qin, Y. Bai, Y. Zhang, and Y. Fu, "Recent advances on neural network pruning at initialization," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 5638–5645.

[37] Y. He and L. Xiao, "Structured pruning for deep convolutional neural networks: A survey," 2023, *arXiv:2303.00566*.

[38] G. Fang, X. Ma, M. Song, M. B. Mi, and X. Wang, "DepGraph: Towards any structural pruning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 16091–16101.

[39] G. Fang, X. Ma, and X. Wang, "Structural pruning for diffusion models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, Art. no. 731.

[40] X. Zhu, J. Li, Y. Liu, C. Ma, and W. Wang, "A survey on model compression for large language models," 2023, *arXiv:2308.07633*.

[41] R. Reed, "Pruning algorithms-A survey," *IEEE Trans. Neural Netw.*, vol. 4, no. 5, pp. 740–747, Sep. 1993.

[42] D. Blalock, J. J. G. Ortiz, J. Frankle, and J. Guttag, "What is the state of neural network pruning?," in *Proc. Mach. Learn. Syst.*, 2020, pp. 129–146.

[43] J. T. O'Neill, "An survey of neural network compression," 2020, *arXiv: 2006.03669*.

[44] J. Liu, S. Tripathi, U. Kurup, and M. Shah, "Pruning algorithms to accelerate convolutional neural networks for edge applications: A survey," 2020, *arXiv: 2005.04275*.

[45] T. Choudhary, V. Mishra, A. Goswami, and J. Sarangapani, "A comprehensive survey on model compression and acceleration," *Artif. Intell. Rev.*, vol. 53, pp. 5113–5155, 2020.

[46] T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden, and A. Peste, "Sparsity in deep learning: Pruning and growth for efficient inference and training in neural networks," *J. Mach. Learn. Res.*, vol. 22, 2021, Art. no. 241.

[47] W. Wang et al., "Model compression and efficient inference for large language models: A survey," 2024, *arXiv:2402.09748*.

[48] J. Frankle and M. Carbin, "The lottery ticket hypothesis: Finding sparse, trainable neural networks," in *Proc. Int. Conf. Learn. Representations*, 2019.

[49] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, "A simple and effective pruning approach for large language models," in *Proc. Int. Conf. Learn. Representations*, 2024.

[50] H. Tanaka, D. Kunin, D. L. Yamins, and S. Ganguli, "Pruning neural networks without any data by iteratively conserving synaptic flow," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 6377–6389.

[51] F. Meng et al., "Pruning filter in filter," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1479.

[52] X. Ma et al., "An image enhancing pattern-based sparsity for real-time inference on mobile devices," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 629–645.

[53] H. Wang and Y. Fu, "Trainability preserving neural structured pruning," in *Proc. Int. Conf. Learn. Representations*, 2023.

[54] N. Lee, T. Ajanthan, and P. H. Torr, "SNIP: Single-shot network pruning based on connection sensitivity," in *Proc. Int. Conf. Learn. Representations*, 2019.

[55] J. Su et al., "Sanity-checking pruning methods: Random tickets can win the jackpot," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 20390–20401.

[56] C. Wang, G. Zhang, and R. Grosse, "Picking winning tickets before training by preserving gradient flow," in *Proc. Int. Conf. Learn. Representations*, 2020.

[57] Z. Liu, J. Li, Z. Shen, G. Huang, Shoumeng, and Z. Changshui, "Learning efficient convolutional networks through network slimming," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 2736–2744.

[58] W. Wen, C. Wu, Y. Wang, Y. Chen, and H. Li, "Learning structured sparsity in deep neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 2074–2082.

[59] Z. Huang and N. Wang, "Data-driven sparse structure selection for deep neural networks," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 304–320.

[60] H. Yang, H. Yin, and M. Shen, P. Molchanov, H. Li, and J. Kautz, "Global vision transformer pruning with Hessian-aware saliency," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 18547–18557.

[61] J. Liu et al., "Dynamic sparse training: Find efficient sparse network from scratch with trainable masked layers," in *Proc. Int. Conf. Learn. Representations*, 2020.

[62] L. Liu et al., "Group fisher pruning for practical network compression," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 7021–7032.

[63] H. Li, A. Kadav, I. Durdanovic, H. Samet, and H. P. Graf, "Pruning filters for efficient ConvNets," in *Proc. Int. Conf. Learn. Representations*, 2017.

[64] Y. Tang, Y. Wang, Y. Deng, C. Xu, D. Tao, and C. Xu, "Manifold regularized dynamic network pruning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 5018–5028.

[65] X. Men et al., "ShortGPT: Layers in large language models are more redundant than you expect," 2024, *arXiv:2403.03853*.

[66] A. Vaswani et al., "Attention is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.

[67] M. Nonnenmacher, T. Pfeil, I. Steinwart, and D. Reeb, "SOSP: Efficiently capturing global correlations by second-order structured pruning," in *Proc. Int. Conf. Learn. Representations*, 2022.

[68] Z. Li et al., "NPAS: A compiler-aware framework of unified network pruning and architecture search for beyond real-time mobile acceleration," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 14255–14266.

[69] S. Liu et al., "The unreasonable effectiveness of random pruning: Return of the most naive baseline for sparse training," in *Proc. Int. Conf. Learn. Representations*, 2022.

[70] Y. Wang et al., "Pruning from scratch," in *Proc. AAAI Conf. Artif. Intell.*, 2020, pp. 12273–12280.

[71] Y. Bai, H. Wang, Z. Tao, K. Li, and Y. Fu, "Dual lottery ticket hypothesis," in *Proc. Int. Conf. Learn. Representations*, 2022.

[72] N. Lee, T. Ajanthan, S. Gould, and P. H. Torr, "A signal propagation perspective for pruning neural networks at initialization," in *Proc. Int. Conf. Learn. Representations*, 2020.

[73] A. M. Saxe, J. L. McClelland, and S. Ganguli, "Exact solutions to the nonlinear dynamics of learning in deep linear neural networks," in *Proc. Int. Conf. Learn. Representations*, 2014.

[74] T. Gebhart, U. Saxena, and P. Schrater, "A unified paths perspective for pruning at initialization," 2021, *arXiv:2101.10552*.

[75] A. Jacot, F. Gabriel, and C. Hongler, "Neural tangent kernel: Convergence and generalization in neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 8580–8589.

[76] J. Frankle, G. K. Dziugaite, D. M. Roy, and M. Carbin, "Pruning neural networks at initialization: Why are we missing the mark?," in *Proc. Int. Conf. Learn. Representations*, 2021.

[77] A. Gaier and D. Ha, "Weight agnostic neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 5365–5379.

[78] V. Ramanujan, M. Wortsman, A. Kembhavi, A. Farhadi, and M. Rastegari, "What's hidden in a randomly weighted neural network?," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11890–11899.

[79] D. Hoang et al., "Revisiting pruning at initialization through the lens of Ramanujan graph," in *Proc. Int. Conf. Learn. Representations*, 2023.

[80] T. Kumar, K. Luo, and M. Sellke, "No free prune: Information-theoretic barriers to pruning at initialization," 2024, *arXiv:2402.01089*.

[81] U. Evci, T. Gale, J. Menick, P. S. Castro, and E. Elsen, "Rigging the lottery: Making all tickets winners," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 2943–2952.

[82] C. Zhao, B. Ni, J. Zhang, Q. Zhao, W. Zhang, and Q. Tian, "Variational convolutional neural network pruning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2780–2789.

[83] Y. He, G. Kang, X. Dong, Y. Fu, and Y. Yang, "Soft filter pruning for accelerating deep convolutional neural networks," in *Proc. Int. Joint Conf. Artif. Intell.*, 2018, pp. 2234–2240.

[84] Z. Liu et al., "MetaPruning: Meta learning for automatic neural network channel pruning," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2019, pp. 3295–3304.

[85] X. Ning, T. Zhao, W. Li, P. Lei, Y. Wang, and H. Yang, "DSA: More efficient budgeted pruning via differentiable sparsity allocation," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 592–607.

[86] M. Cho, S. Adya, and D. Naik, "PDP: Parameter-free differentiable pruning is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2023, Art. no. 1986.

[87] M. Yuan and Y. Lin, "Model selection and estimation in regression with grouped variables," *J. Roy. Statist. Soc.: Ser. B (Statist. Methodol.)*, vol. 68, pp. 49–67, 2006.

[88] A. Gordon, E. Eban, O. Nachum, and B. Chen, "MorphNet: Fast & simple resource-constrained structure learning of deep networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 1586–1595.

[89] T. Li, B. Wu, Y. Yang, Y. Fan, Y. Zhang, and W. Liu, "Compressing convolutional neural networks via factorized convolutional filters," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 3977–3986.

[90] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, "Distributed optimization and statistical learning via the alternating direction method of multipliers," *Found. Trends Mach. Learn.*, vol. 3, no. 1, pp. 1–122, 2010.

[91] D. C. Mocanu, E. Mocanu, P. Stone, P. H. Nguyen, M. Gibescu, and A. Liotta, "Scalable training of artificial neural networks with adaptive sparse connectivity inspired by network science," *Nature Commun.*, vol. 9, no. 1, 2018, Art. no. 2383.

[92] S. Liu et al., "Sparse training via boosting pruning plasticity with neuroregeneration," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 9908–9922.

[93] S. Liu et al., "Deep ensembling with no overhead for either training or testing: The all-round blessings of dynamic sparsity," in *Proc. Int. Conf. Learn. Representations*, 2022.

[94] X. Dai, H. Yin, and N. K. Jha, "NeST: A neural network synthesis tool based on a grow-and-prune paradigm," *IEEE Trans. Comput.*, vol. 68, no. 10, pp. 1487–1497, Oct. 2019.

[95] H. Mostafa and X. Wang, "Parameter efficient training of deep convolutional neural networks by dynamic sparse reparameterization," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 4646–4655.

[96] T. Lin et al., "Dynamic model pruning with feedback," in *Proc. Int. Conf. Learn. Representations*, 2020.

[97] T. Dettmers and L. Zettlemoyer, "Sparse networks from scratch: Faster training without losing performance," 2019, *arXiv: 1907.04840*.

[98] G. Sokar et al., "Dynamic sparse training for deep reinforcement learning," in *Proc. Int. Joint Conf. Artif. Intell.*, 2022, pp. 3437–3443.

[99] U. Evci, Y. A. Ioannou, C. Keskin, and Y. Dauphin, "Gradient flow in sparse neural networks and how lottery tickets win," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 6577–6586.

[100] Y. He, P. Liu, Z. Wang, Z. Hu, and Y. Yang, "Filter pruning via geometric median for deep convolutional neural networks acceleration," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 4340–4349.

[101] P. Fletcher, S. Venkatasubramanian, and S. Joshi, "Robust statistics on Riemannian manifolds via the geometric median," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2008, pp. 1–8.

[102] S. Guo, Y. Wang, Q. Li, and J. Yan, "DMCP: Differentiable Markov channel pruning for neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1536–1544.

[103] M. Ma, J. Wang, and Z. Yu, "Differentiable network pruning via polarization of probabilistic channelwise soft masks," *Comput. Intell. Neurosci.*, vol. 2022, 2022, Art. no. 7775419.

[104] L. Yu and W. Xiang, "X-pruner: Explainable pruning for vision transformers," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 24355–24363.

[105] X. Ding, G. Ding, J. Han, and S. Tang, "Auto-balanced filter pruning for efficient convolutional neural networks," in *Proc. AAAI Conf. Artif. Intell.*, 2018, pp. 6797–6804.

[106] P. Molchanov, A. Mallya, S. Tyree, I. Frosio, and J. Kautz, "Importance estimation for neural network pruning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11264–11272.

[107] S. Lin, R. Ji, Y. Li, C. Deng, and X. Li, "Towards compact ConvNets via structure-sparsity regularized filter pruning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 2, pp. 574–588, Feb. 2020.

[108] H. Wang, C. Qin, Y. Zhang, and Y. Fu, "Neural pruning via growing regularization," in *Proc. Int. Conf. Learn. Representations*, 2021.

[109] X. Chen, Y. Hu, and J. Zhang, "Compressing large language models by streamlining the unimportant layer," 2024, *arXiv:2403.19135*.

[110] Z. Liu, M. Sun, T. Zhou, G. Huang, and T. Darrell, "Rethinking the value of network pruning," in *Proc. Int. Conf. Learn. Representations*, 2019.

[111] A. Renda, J. Frankle, and M. Carbin, "Comparing rewinding and fine-tuning in neural network pruning," in *Proc. Int. Conf. Learn. Representations*, 2020.

[112] B. Liu et al., "A survey of lottery ticket hypothesis," 2024, *arXiv:2403.04861*.

[113] J. Diffenderfer and B. Kailkhura, "Multi-prize lottery ticket hypothesis: Finding a accurate binary neural networks by pruning a randomly weighted network," in *Proc. Int. Conf. Learn. Representations*, 2021.

[114] X. Chen, Z. Zhang, Y. Sui, and T. Chen, "GANs can play lottery tickets too," in *Proc. Int. Conf. Learn. Representations*, 2021.

[115] N. M. Kalibhat, Y. Balaji, and S. Feizi, "Winning lottery tickets in deep generative models," in *Proc. AAAI Conf. Artif. Intell.*, 2021, pp. 8038–8046.

[116] B. Hui, D. Yan, X. Ma, and W.-S. Ku, "Rethinking graph lottery tickets: Graph sparsity matters," in *Proc. Int. Conf. Learn. Representations*, 2023.

[117] S. Prasanna, A. Rogers, and A. Rumshisky, "When BERT plays the lottery, all tickets are winning," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2020, pp. 3208–3229.

[118] E. Malach, G. Yehudai, S. Shalev-Shwartz, and O. Shamir, "Proving the lottery ticket hypothesis: Pruning is all you need," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 6682–6691.

[119] L. Orseau, M. Hutter, and O. Rivasolata, "Logarithmic pruning is all you need," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 246.

[120] A. S. Morcos, H. Yu, M. Paganini, and Y. Tian, "One ticket to win them all: Generalizing lottery ticket initializations across datasets and optimizers," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 4933–4943.

[121] R. Mehta, "Sparse transfer learning via winning lottery tickets," 2019, *arXiv: 1905.07785*.

[122] Z. Gan et al., "Playing lottery tickets with vision and language," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 652–660.

[123] T. Chen et al., "A unified lottery ticket hypothesis for graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 1695–1706.

[124] Z. Zhang et al., "Validating the lottery ticket hypothesis with inertial manifold theory," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 30196–30210.

[125] S. Zhang, M. Wang, S. Liu, P.-Y. Chen, and J. Xiong, "Why lottery ticket wins? a theoretical perspective of sample complexity on pruned neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, Art. no. 207.

[126] M. Paul et al., "Unmasking the lottery ticket hypothesis-what's encoded in a winning ticket's mask," in *Proc. Int. Conf. Learn. Representations*, 2023.

[127] X. Ma et al., "Sanity checks for lottery tickets: Does your winning ticket really win the jackpot?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 12749–12760.

[128] N. Liu et al., "Lottery ticket preserves weight correlation: Is it desirable or not?," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 7011–7020.

[129] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2022.

[130] D. Shi, C. Tao, Y. Jin, Z. Yang, C. Yuan, and J. Wang, "UPop: Unified and progressive pruning for compressing vision-language transformers," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 31292–31311.

[131] A. Nova, H. Dai, and D. Schuurmans, "Gradient-free structured pruning with unlabeled data," in *Proc. Int. Conf. Mach. Learn.*, 2023, pp. 26326–26341.

[132] L. Dery et al., "Everybody prune now: Structured pruning of LLMs with only forward passes," 2024, *arXiv:2402.05406*.

[133] B.-K. Kim et al., "Shortened LLaMA: A simple depth pruning for large language models," 2024, *arXiv:2402.02834*.

[134] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2017, pp. 1398–1406.

[135] S. Lin et al., "Towards optimal structured CNN pruning via generative adversarial learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2790–2799.

[136] M. Xia, Z. Zhong, and D. Chen, "Structured pruning learns compact and accurate models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2022, pp. 1513–1528.

[137] M. Xia, T. Gao, Z. Zeng, and D. Chen, "Sheared LLaMA: Accelerating language model pre-training via structured pruning," in *Proc. Int. Conf. Learn. Representations*, 2024.

[138] R. Tibshirani, "Regression shrinkage and selection via the lasso," *J. Roy. Statist. Soc.*, vol. 58, no. 1, pp. 267–288, 1996.

[139] H. Yang, Y. Zhu, and J. Liu, "ECC: Platform-independent energy-constrained deep neural network compression via a bilinear regression model," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11206–11215.

[140] S. Gao, F. Huang, W. Cai, and H. Huang, "Network pruning via performance maximization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 9270–9280.

[141] X. Chen, Y. Cheng, S. Wang, Z. Gan, Z. Wang, and J. Liu, "EarlyBERT: Efficient BERT training via early-bird lottery tickets," in *Proc. Annu. Meeting Assoc. Comput. Linguistics 11th Int. Joint Conf. Natural Lang. Process.*, 2021, pp. 2195–2207.

[142] P. Michel, O. Levy, and G. Neubig, "Are sixteen heads really better than one?," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 14014–14024.

[143] J. Frankle et al., "Linear mode connectivity and the lottery ticket hypothesis," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 3259–3269.

[144] W. Kwon, S. Kim, M. W. Mahoney, J. Hassoun, K. Keutzer, and A. Gholami, "A fast post-training pruning framework for transformers," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, Art. no. 1750.

[145] Y. An et al., "Fluctuation-based adaptive structured pruning for large language models," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 10865–10873.

[146] Y. Rao, J. Lu, J. Lin, and J. Zhou, "Runtime network routing for efficient image classification," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 41, no. 10, pp. 2291–2304, Oct. 2019.

[147] W. Hua, Y. Zhou, C. D. Sa, Z. Zhang, and G. E. Suh, "Channel gating neural networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 1886–1896.

[148] X. Gao et al., "Dynamic channel pruning: Feature boosting and suppression," in *Proc. Int. Conf. Learn. Representations*, 2019.

[149] S. Elkerdawy, M. Elhoushi, H. Zhang, and N. Ray, "Fire together wire together: A dynamic pruning approach with self-supervised mask prediction," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12454–12463.

[150] J. Meng, L. Yang, J. Shin, D. Fan, and J. Sun Seo, "Contrastive dual gating: Learning sparse features with contrastive learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12247–12255.

[151] R. Hadsell, S. Chopra, and Y. LeCun, "Dimensionality reduction by learning an invariant mapping," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2006, pp. 1735–1742.

[152] S. Tuli and N. K. Jha, "AccelTran: A sparsity-aware accelerator for dynamic inference with transformers," *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.*, vol. 42, no. 11, pp. 4038–4051, Nov. 2023.

[153] A. See, M.-T. Luong, and C. D. Manning, "Compression of neural machine translation models via pruning," 2016, *arXiv:1606.09274*.

[154] E. S. Lubana and R. P. Dick, "A gradient flow framework for analyzing network pruning," in *Proc. Int. Conf. Learn. Representations*, 2021.

[155] V. Sehwag et al., "Towards compact and robust deep neural networks," 2019, *arXiv: 1906.06110*.

[156] P. Molchanov, S. Tyree, T. Karras, T. Aila, and J. Kautz, "Pruning convolutional neural networks for resource efficient inference," in *Proc. Int. Conf. Learn. Representations*, 2017.

[157] Y. Li et al., "Towards compact CNNs via collaborative compression," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 6438–6447.

[158] F. Yu, K. Huang, M. Wang, Y. Cheng, W. Chu, and L. Cui, "Width & depth pruning for vision transformers," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 3143–3151.

[159] J. Rosenfeld, J. Frankle, M. Carbin, and N. Shavit, "On the predictability of pruning across scales," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 9075–9083.

[160] J. Lee, S. Park, S. Mo, S. Ahn, and J. Shin, "Layer-adaptive sparsity for the magnitude-based pruning," in *Proc. Int. Conf. Learn. Representations*, 2021.

[161] Y. LeCun, J. Denker, and S. Solla, "Optimal brain damage," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1989, pp. 598–605.

[162] M. Santacroce et al., "What matters in the structured pruning of generative language models?," 2023, *arXiv:2302.03773*.

[163] T. F. A. van der Ouderaa et al., "The LLM surgeon," 2024, *arXiv:2312.17244*.

[164] B. Hassibi and D. G. Stork, "Second order derivatives for network pruning: Optimal brain surgeon," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 1992, pp. 164–171.

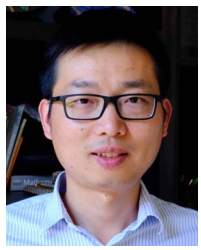
[165] C. Wang et al., "EigenDamage: Structured pruning in the Kronecker-Factored eigenbasis," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 6566–6575.

[166] E. Kurtic et al., "The optimal BERT surgeon: Scalable and accurate second-order pruning for large language models," in *Proc. Conf. Empir. Methods Natural Lang. Process.*, 2022, pp. 4163–4181.
[167] Y. Li, S. Gu, K. Zhang, L. Van Gool, and R. Timofte, "DHP: Differentiable meta pruning via hypernetworks," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 608–624.
[168] E. Voita, D. Talbot, F. Moiseev, R. Sennrich, and I. Titov, "Analyzing multi-head self-attention: Specialized heads do the heavy lifting, the rest can be pruned," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2019, pp. 5797–5808.
[169] M. Zhang, X. Yu, J. Rong, and L. Ou, "Graph pruning for model compression," *Appl. Intell.*, vol. 52, pp. 11244–11256, 2022.
[170] Y. He, J. Lin, Z. Liu, H. Wang, L.-J. Li, and S. Han, "AMC: AutoML for model compression and acceleration on mobile devices," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 784–800.
[171] Z. Zhong, J. Yan, W. Wu, J. Shao, and C.-L. Liu, "Practical block-wise neural network architecture generation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 2423–2432.
[172] J. V. Amersfoort et al., "Single shot structured pruning before training," 2020, *arXiv: 2007.00389*.
[173] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," 2016, *arXiv:1609.07843*.
[174] S. Zhang et al., "OPT: Open pre-trained transformer language models," 2022, *arXiv:2205.01068*.
[175] F. J. Huang and Y. LeCun, "Learning methods for generic object recognition with invariance to pose and lighting," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2004, pp. 97–104.
[176] A. Krizhevsky, "Learning multiple layers of features from tiny images," 2009. [Online]. Available: https://www.cs.toronto.edu/kriz/learning-features-2009-TR.pdf
[177] H. Touvron et al., "LLaMA: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.
[178] M. Marcus, B. Santorini, and M. A. Marcinkiewicz, "Building a large annotated corpus of English: The Penn Treebank," *Comput. Linguistics*, vol. 19, pp. 313–330, 1993.
[179] Y. Li et al., "Exploiting kernel sparsity and entropy for interpretable CNN compression," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 2800–2809.
[180] O. Russakovsky et al., "ImageNet large scale visual recognition challenge," *Int. J. Comput. Vis.*, vol. 115, pp. 211–252, 2015.
[181] M. Zhang et al., "LoRAPrune: Pruning meets low-rank parameter-efficient fine-tuning," 2023, *arXiv:2305.18403*.
[182] S. Singh and R. Liu, "Why is pruning at initialization immune to reinitializing and shuffling?," 2021, *arXiv:2107.01808*.
[183] Y. Qiu and R. Suda, "Train-by-Reconnect: Decoupling locations of weights from their values," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1759.
[184] C. R. Wolfe, Q. Wang, J. L. Kim, and A. Kyrillidis, "How much pre-training is enough to discover a good subnetwork?," *Trans. Mach. Learn. Res.*, vol. 2024, pp. 1–34, 2024.
[185] I. Amos, J. Berant, and A. Gupta, "Never train from scratch: Fair comparison of long-sequence models requires data-driven priors," in *Proc. Int. Conf. Learn. Representations*, 2024.
[186] T.-W. Chin, R. Ding, C. Zhang, and D. Marculescu, "Towards efficient model compression via learned global ranking," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 1518–1528.
[187] R. Tiwari, U. Bamba, A. Chavan, and D. K. Gupta, "ChipNet: Budget-aware pruning with heaviside continuous approximations," in *Proc. Int. Conf. Learn. Representations*, 2021.
[188] O. Kovaleva et al., "BERT Busters: Outlier dimensions that disrupt transformers," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2021, pp. 3392–3405.
[189] G. Bai, Y. Li, C. Ling, K. Kim, and L. Zhao, "SparseLLM: Towards global pruning for pre-trained language models," 2024, *arXiv:2402.17946*.
[190] D. H. Le and B.-S. Hua, "Network pruning that matters: A case study on pretraing variants," in *Proc. Int. Conf. Learn. Representations*, 2021.
[191] T. Zhang et al., "A systematic DNN weight pruning framework using alternating direction method of multipliers," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 191–207.
[192] M. Ye, C. Gong, L. Nie, D. Zhou, A. Klivans, and Q. Liu, "Good subnetworks provably exist: Pruning via greedy forward selection," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 10820–10830.
[193] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, "Training data-efficient image transformers & distillation through attention," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 10347–10357.
[194] S. J. Pan and Q. Yang, "A suvery on tranfer learning," *IEEE Trans. Knowl. Data Eng.*, vol. 22, pp. 1345–1359, Oct. 2010.
[195] T. Chen et al., "The lottery ticket hypothesis for pre-trained BERT networks," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1328.
[196] T. Chen et al., "The lottery tickets hypothesis for supervised and self-supervised pre-training in computer vision models," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 16306–16316.
[197] E. Iofinova, A. Peste, M. Kurtz, and D. Alistarh, "How well do sparse ImageNet models transfer?," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 12256–12266.
[198] X. Chen, Y. Cheng, S. Wang, Z. Gan, J. Liu, and Z. Wang, "The elastic lottery ticket hypothesis," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 26609–26621.
[199] F. Li, G. Li, X. He, and J. Cheng, "Dynamic dual gating neural networks," in *Proc. IEEE Int. Conf. Comput. Vis.*, 2021, pp. 5310–5319.
[200] V. Sanh, T. Wolf, and A. M. Rush, "Movement pruning: Adaptive sparsity by fine-tuning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2020, Art. no. 1711.
[201] C. Zhang, S. Bengio, and Y. Singer, "Are all layers created equal?," *J. Mach. Learn. Res.*, vol. 23, 2022, Art. no. 67.
[202] R. Gong et al., "Fast and controllable post-training sparsity-learning optimal sparsity allocation with global constraint in minutes," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 12190–12198.
[203] L. Yin et al., "Outlier weighed layerwise sparsity (OWL): A missing secret sauce for pruning LLMs to high sparsity," 2024, *arXiv:2310.05175*.
[204] S. Yuan, E. Nie, B. Ma, and M. Färber, "Why lift so heavy? Slimming large language models by cutting off the layers," 2024, *arXiv:2402.11700*.
[205] Y. Yang, Z. Cao, and H. Zhao, "LaCo: Large language model pruning via layer collapse," 2024, *arXiv:2402.11187*.
[206] S. Kalyan, S. Joshi, S. Sheik, B. U. Pedroni, and G. Cauwcnbcrghs, "Unsupervised synaptic pruning strategies for restricted Boltzmann machines," in *Proc. IEEE Biomed. Circuits Syst. Conf.*, 2018, pp. 1–4.
[207] X. Liu et al., "Self-supervised learning: Generative or contrastive," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 1, pp. 857–876, Jan. 2023.
[208] M. Caron, A. Morcos, P. Bojanowski, J. Mairal, and A. Joulin, "Pruning convolutional neural networks with self-supervision," 2020, *arXiv: 2001.03554*.
[209] S. Pan, Y. Qin, T. Li, X. Li, and L. Hou, "Momentum contrastive pruning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2022, pp. 2647–2656.
[210] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
[211] X. Chen, H. Fan, R. Girshick, and K. He, "Improved baselines with momentum contrastive learning," 2020, *arXiv: 2003.04297*.
[212] C.-I. Jeff Lai et al., "PARP: Prune, adjust and re-prune for self-supervised speech recognition," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2021, pp. 21256–21272.
[213] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
[214] F. Tung and G. Mori, "CLIP-Q: Deep network compression learning by in-parallel pruning-quantization," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2018, pp. 7873–7882.
[215] Y. Li, S. Gu, C. Mayer, L. V. Gool, and R. Timofte, "Group sparsity: The hinge between filter pruning and decomposition for network compression," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 8018–8027.
[216] Y. Li et al., "LoSparse: Structured compression of large language models based on low-rank and sparse approximation," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2023, pp. 20336–20350.
[217] X. Dong and Y. Yang, "Network pruning via transformable architecture search," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2019, pp. 759–770.
[218] A. Klein, J. Golebiowski, X. Ma, V. Perrone, and C. Archambeau, "Structural pruning of large language models via neural architecture search," in *Proc. Int. Conf. Autom. Mach. Learn.*, 2023.
[219] Y. Liu, Z. Shu, Y. Li, Z. Lin, F. Perazzi, and S. Kung, "Content-aware GAN compression," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 12156–12166.
[220] J. Park and A. No, "Prune your model before distill it," in *Proc. Eur. Conf. Comput. Vis.*, 2022, pp. 120–136.

[221] L. Chen, Y. Chen, J. Xi, and X. Le, "Knowledge from the original network: Restore a better pruned network with knowledge distillation," *Complex Intell. Syst.*, vol. 8, pp. 709–718, 2022.
[222] T. Chen et al., "Long live the lottery: The existence of winning tickets in lifelong learning," in *Proc. Int. Conf. Learn. Representations*, 2021.
[223] Y. Zhang et al., "Advancing model pruning via bi-level optimization," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 18309–18326.
[224] W. Zou, Y. Wang, X. Fu, and Y. Cao, "Dreaming to prune image deraining networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 6023–6032.
[225] M. Wang et al., "Large multimodal model compression via efficient pruning and distillation at AntGroup," 2023, *arXiv:2312.05795*.
[226] Y. Mao et al., "LadaBERT: Lightweight adaptation of BERT through hybrid model compression," in *Proc. Int. Conf. Comput. Linguistics*, 2020, pp. 3225–3234.
[227] L. Yao, R. Pi, H. Xu, W. Zhang, Z. Li, and T. Zhang, "Joint-DetNAS: Upgrade your detector with NAS, pruning and dynamic distillation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 1899–1908.
[228] H. Wang, S. Gui, H. Yang, J. Liu, and Z. Wang, "GAN slimming: All-in-one GAN compression by a unified optimization framework," in *Proc. Eur. Conf. Comput. Vis.*, 2020, pp. 54–73.
[229] Z. Li et al., "Train large, then compress: Rethinking model size for efficient training and inference of transformers," in *Proc. Int. Conf. Mach. Learn.*, 2020, Art. no. 553.
[230] K. Han, Y. Wang, J. Guo, and E. Wu, "ParameterNet: Parameters are all you need," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2024, pp. 15751–15761.
[231] Y. Chen, X. Dai, M. Liu, D. Chen, L. Yuan, and Z. Liu, "Dynamic convolution: Attention over convolution kernels," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 11030–11039.
[232] Q. Yan, D. Gong, Y. Liu, A. van den Hengel, and J. Q. Shi, "Learning Bayesian sparse networks with full experience replay for continual learning," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 109–118.
[233] F. Corti, R. Entezari, S. Hooker, D. Bacciu, and O. Saukh, "Studying the impact of magnitude pruning on contrastive learning methods," in *Proc. Int. Conf. Mach. Learn.*, 2022.
[234] Y. Jiang et al., "Model pruning enables efficient federated learning on edge devices," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10374–10386, Dec. 2023.
[235] X. Sui et al., "A hardware-friendly high-precision CNN pruning method and its FPGA implementation," *Sensors*, vol. 23, 2023.
[236] OpenAI, J. Achiam and S. Adler, "GPT-4 technical report," 2023, *arXiv:2303.08774*.
[237] H. Wang, C. Qin, Y. Bai, and Y. Fu, "Why is the state of neural network pruning so confusing? On the fairness, comparison setup, and trainability in network pruning," 2023, *arXiv:2301.05219*.

Hongrong Cheng received the PhD degree in information and communication engineering from the University of Electronic Science and Technology of China. She is now working toward the second PhD degree in computer science with the University of Adelaide. Her primary research focuses on model compression.

Miao Zhang (Member, IEEE) received the PhD degree from the University of Technology Sydney (UTS), Australia. He is currently a professor with the Harbin Institute of Technology (Shenzhen), Shenzhen, China. Before that, he was an assistant professor with Aalborg University, Aalborg, Denmark. His primary research interests include AutoML, model compression, and continual learning.



Javen Qinfeng Shi (Member, IEEE) is a professor with the School of Computer and Mathematical Sciences, University of Adelaide, is founding director of Causal AI Group, and director of Advanced Reasoning and Learning of Australian Institute for Machine Learning (AIML). His core expertise includes Probabilistic Graphical Models, Causation, and Deep Learning. He has transferred his research to diverse industries including agriculture, mining, sport, manufacturing, bushfire, material discovery, health, and education.